

UNIVERSIDADE PAULISTA

CAMPUS – PARAÍSO

TURNO – MATUTINO

Amanda Justino Oliveira - RA: R093689

Breno Faustino Almeida - RA: G972732

Bruno Alves de Sousa - RA: G975BG3

Gabriel Santana Cordeiro - RA: G972716

Maria Eduarda Moreira - RA: R0726G9

Pedro Henrique Gomes dos Santos - RA: R085287

**DESENVOLVIMENTO DE UM SISTEMA PARA ANÁLISE DE
PERFORMANCE DE ALGORITMOS DE ORDENAÇÃO DE DADOS**

SÃO PAULO

2025

UNIVERSIDADE PAULISTA

CAMPUS – PARAÍSO

TURNO – MATUTINO

Amanda Justino Oliveira - RA: R093689

Breno Faustino Almeida - RA: G972732

Bruno Alves de Sousa - RA: G975BG3

Gabriel Santana Cordeiro - RA: G972716

Maria Eduarda Moreira - RA: R0726G9

Pedro Henrique Gomes dos Santos - RA: R085287

**DESENVOLVIMENTO DE UM SISTEMA PARA ANÁLISE DE
PERFORMANCE DE ALGORITMOS DE ORDENAÇÃO DE DADOS**

Atividade prática supervisionada
(APS) apresentada para a disciplina de
Estruturas de Dados.

Orientador: Prof. Roberto Leminski

SÃO PAULO

2025

SUMÁRIO

1	OBJETIVO DO TRABALHO.....	3
2	INTRODUÇÃO.....	4
3	REFERENCIAL TEÓRICO	6
3.1	Conceitos de Estruturas de Dados	6
3.1.1	Alocação de Memória.....	7
3.1.2	Tipos de Dados em C.....	7
3.1.3	Tipos Abstratos.....	8
3.2	Bubble Sort.....	8
3.3	Selection Sort.....	9
3.4	Insertion Sort.....	10
3.5	Quick Sort.....	11
3.6	Merge Sort	12
3.7	Shell Sort	13
3.8	Cocktail Shaker Sort.....	14
3.9	Heap Sort.....	15
3.10	Machine Learning	16
3.11	Deep Learning	17
3.12	Técnicas de Deep Learning aplicadas à imagem.....	18
4	METODOLOGIA	21
4.1	Tipo de Pesquisa e Fundamentação Teórica.....	21
4.2	Ambiente de Desenvolvimento e Ferramentas	21
4.3	Base de Dados	22
4.4	CrITÉrios de Ordenação Implementados:	23
4.5	Implementação dos Algoritmos de Ordenação.....	23
4.6	Bibliotecas utilizadas	24
4.7	CrITÉrio de Validação	25
4.8	Estratégia de Testes	25
5	PROJETO/ESTRUTURA DO PROGRAMA.....	27
5.1	Bibliotecas.....	27
5.2	Bubble Sort.....	28
5.2	Cocktail Shaker Sort.....	30

5.3	Insertion Sort.....	34
5.4	Shell Sort	36
5.5	Heap Sort.....	38
5.6	Selection Sort.....	40
5.7	Quick Sort.....	42
5.8	Merge Sort	45
5.9	Função Main	49
6	CONCLUSÃO	57
6.1	Discussão dos Resultados Obtidos	57
6.2	Impacto do Algoritmo de Ordenação para Visualização de Fenômenos do Meio Ambiente	58
6.3	Relação de Machine e Deep Learning com Algoritmos de Ordenação	59
6.4	Importância dos Algoritmos de Ordenação para a Ciência da Computação	61
6.5	Considerações Finais	61
7	RELATÓRIO COM AS LINHAS DE CÓDIGO DO PROGRAMA	64
	REFERÊNCIAS	85
8	FICHAS DE ATIVIDADES PRÁTICAS SUPERVISIONADAS	89

1 OBJETIVO DO TRABALHO

O objetivo deste trabalho tem por vez desenvolver e implementar um framework computacional integrado para análise de performance de algoritmos de processamento e ordenação aplicados ao geoprocessamento de dados de focos de incêndio detectados por satélites em território nacional, estabelecendo um processo eficiente que possibilite a classificação automática de áreas desmatadas e a geração de análises preditivas sobre padrões de desmatamento.

Este trabalho também visa a introdução para áreas de Inteligência Artificial, como Machine Learning e Deep Learning, que são ferramentas de extrema importância, capazes de processar, analisar e extrair dados específicos de uma grande quantidade de informação. Os algoritmos de ML e DL aprendem padrões diretamente dos dados, sem necessidade de programação explícita para cada cenário.

Além disso, o trabalho propõe uma abordagem multidisciplinar, integrando conceitos de programação, estruturas de dados, geoprocessamento e inteligência artificial. Essa integração possibilita uma compreensão mais ampla sobre a aplicação prática dos algoritmos de ordenação e análise de dados em contextos reais, especialmente no monitoramento e na gestão ambiental. Dessa forma, o projeto não apenas promove o desenvolvimento técnico dos envolvidos, como também evidencia o papel da tecnologia como ferramenta essencial para apoiar pesquisas, análises e tomadas de decisão voltadas à sustentabilidade e à preservação dos recursos naturais. Ademais, busca-se elaborar uma dissertação que sintetize os principais tópicos abordados na disciplina ao longo do semestre, demonstrando sua aplicação prática no desenvolvimento e implementação do sistema proposto.

2 INTRODUÇÃO

Na ciência da computação, os algoritmos de ordenação são essenciais, pois servem como base para uma variedade de operações relacionadas à organização, busca e processamento de dados. De acordo com Cormen et al. (2009), entender os métodos de ordenação é fundamental para melhorar o desempenho dos programas e diminuir o tempo de execução de tarefas computacionais que lidam com grandes volumes de dados. Esses algoritmos são amplamente empregados em bancos de dados, sistemas de busca, compressão de dados e análise de informações, sendo vistos como um dos fundamentos da eficiência algorítmica.

Em termos gerais, os algoritmos de ordenação podem ser categorizados com base em seu paradigma de operação. Segundo Knuth (1998), as categorias mais relevantes são os algoritmos de divisão e conquista, inserção, seleção e troca. O paradigma de divisão e conquista é frequentemente empregado em algoritmos como Quick Sort e Merge Sort, pois possibilita a decomposição do problema em partes menores e mais simples, que são resolvidas de maneira recursiva até se chegar ao resultado. Essa abordagem leva a um desempenho assintótico eficiente, geralmente na ordem de $O(n \log n)$.

Além da complexidade, a estabilidade é uma característica relevante em algoritmos de ordenação. Um algoritmo é considerado estável quando preserva a ordem relativa de elementos iguais após a ordenação. Conforme Goodrich, Tamassia e Goldwasser (2014), essa propriedade é especialmente importante em aplicações que envolvem múltiplos critérios de ordenação, como sistemas de gerenciamento de dados e planilhas eletrônicas. Nesse contexto, algoritmos como o Merge Sort se destacam por garantirem estabilidade, enquanto o Quick Sort, em sua forma clássica, não possui essa característica.

A escolha do algoritmo mais apropriado depende não apenas de sua eficiência teórica, mas também de aspectos práticos, como o volume de dados e as restrições de hardware. Knuth (1998) enfatiza que, embora o Quick Sort seja amplamente utilizado devido ao seu excelente desempenho médio, em situações onde a previsibilidade é crucial, o Merge Sort pode ser mais vantajoso. Já o Shell Sort, por sua vez, é indicado para listas de tamanho moderado, enquanto algoritmos simples como o Bubble Sort e o Cocktail Shaker Sort são mais adequados para fins didáticos.

Por fim, a evolução dos estudos sobre algoritmos de ordenação reflete a busca contínua por eficiência computacional. Cormen et al. (2009) destacam que, mesmo após décadas de pesquisa, os algoritmos clássicos continuam sendo amplamente utilizados, tanto por sua robustez quanto pela clareza conceitual. Com o avanço das tecnologias e o aumento exponencial dos volumes de dados, compreender o funcionamento e as propriedades dos métodos de ordenação torna-se essencial para o desenvolvimento de sistemas mais rápidos, escaláveis e confiáveis.

3 REFERENCIAL TEÓRICO

Este capítulo aborda sobre conceitos de estrutura de dados, machine learning e deep learning, além de algoritmos de ordenação que é o foco principal do trabalho em estudo, com o intuito de maior embasamento e enriquecimento dele.

3.1 Conceitos de Estruturas de Dados

O conhecimento sobre estruturas de dados é um dos pilares do desenvolvimento de algoritmos eficientes e processamento de grandes volumes de informação. Segundo Tenenbaum, Langsam e Augenstein (2013), uma estrutura de dados pode ser definida como “uma forma particular de organizar e armazenar dados em um computador, de modo que possam ser utilizados de maneira eficiente”. Assim, essas estruturas permitem representar as relações lógicas entre dados de forma coerente, possibilitando que sejam processados e registrados adequadamente pelo computador.

As estruturas de dados desempenham um papel fundamental no desenvolvimento de software, permitindo criar programas que representem de forma clara e relevante os dados de um problema real. Essa abordagem não apenas melhora a legibilidade do código, como também contribui para a criação de procedimentos mais simples e eficientes, resultando em ganhos significativos de desempenho e redução de custos, ou seja, a escolha adequada da estrutura de dados influencia diretamente a complexidade temporal e espacial dos algoritmos, impactando o desempenho global dos sistemas computacionais. Diante disso, compreender as propriedades, vantagens e limitações de cada tipo de estrutura é essencial para o desenvolvimento de soluções otimizadas e voltadas à análise de performance. (CORMEN et al., 2009)

Na linguagem C essa relação entre organização dos dados e eficiência torna-se ainda mais evidente, uma vez que o programador possui controle direto sobre a alocação e manipulação de memória. A linguagem oferece recursos como ponteiros, structs e alocação dinâmica, que permitem a implementação de diferentes estruturas de dados, como listas encadeadas, pilhas, filas e árvores. Dessa forma, compreender como essas estruturas são representadas e gerenciadas em C é fundamental para

desenvolver algoritmos de alto desempenho, capazes de equilibrar tempo de execução e consumo de memória. (BALIEIRO, 2015)

3.1.1 Alocação de Memória

Um aspecto central no uso de estruturas de dados em C é a forma como a memória é alocada. Existem dois tipos principais de alocação: estática e dinâmica. A alocação estática ocorre em tempo de compilação. Quando uma variável ou estrutura é declarada, é necessário informar seu tipo e tamanho, o que faz com que o compilador reserve um espaço fixo na memória antes da execução do programa. Esse espaço permanece reservado até o término da execução. Por exemplo, vetores e matrizes são estruturas alocadas estaticamente: após a compilação, seus tamanhos tornam-se imutáveis durante toda a execução do programa. Já a alocação dinâmica ocorre em tempo de execução. Nesse caso, as variáveis são declaradas sem tamanho definido, e o espaço de memória é reservado apenas quando solicitado por meio de funções da biblioteca padrão, como `malloc()`, `calloc()` ou `realloc()`. Esse tipo de alocação oferece maior flexibilidade, permitindo criar estruturas de dados cujo tamanho pode variar conforme a necessidade do programa, como listas encadeadas, pilhas e filas. (SANTOS, 2024)

3.1.2 Tipos de Dados em C

Na linguagem C, os tipos de dados são classificados em primitivos e não primitivos. Os tipos primitivos incluem `int`, `float`, `double` e `char`, e estão diretamente relacionados à forma como os dados são representados na memória, sendo o alicerce para a criação de estruturas mais complexas.

As estruturas não primitivas, por sua vez, são compostas a partir dos tipos básicos e incluem vetores, matrizes, registros e ponteiros. Com o uso de ponteiros, é possível manipular endereços de memória diretamente, o que possibilita a implementação de estruturas dinâmicas, como listas encadeadas, árvores e grafos. Essa característica confere à linguagem C grande poder e flexibilidade na manipulação de dados.

3.1.3 Tipos Abstratos

O conceito de Tipo Abstrato de Dados (TAD) é amplamente utilizado na implementação de estruturas de dados em C. Um TAD pode ser definido como um conjunto de tipos de dados e um conjunto de procedimentos (funções) que podem ser aplicadas sobre este conjunto de tipos de dados. Desta forma, para que um tipo abstrato de dado seja implementado, são utilizadas as estruturas de dados. As estruturas de dados representam as relações lógicas entre os dados, por exemplo, relação linear ou não linear (hierárquica) e as operações que serão efetuadas sobre estes dados. Na prática, um TAD é implementado por meio de estruturas (struct) e funções que manipulam seus elementos. Essa abordagem permite encapsular os dados e controlar o acesso a eles, de forma que as operações sejam realizadas apenas por meio das funções definidas, garantindo modularidade, segurança e clareza no código. (BALIEIRO, 2015)

3.2 Bubble Sort

Os algoritmos de ordenação são processos que direcionam para a organização, e reorganização, de valores apresentados em uma dada sequência, para que os dados possam ser acessados posteriormente de forma mais eficiente. A ordenação adequada de dados é um passo fundamental para o desempenho de sistemas que realizam operações de busca, comparação e análise, influenciando diretamente na eficiência dos algoritmos e na utilização dos recursos computacionais.

Entre os diversos métodos de ordenação existentes, o Bubble Sort, também conhecido como ordenação por flutuação, destaca-se por sua simplicidade e fácil compreensão. Seu funcionamento baseia-se na comparação entre pares de elementos adjacentes em um vetor. A cada passada, o algoritmo compara dois elementos consecutivos e, caso o primeiro seja maior que o segundo, realiza a troca de suas posições. Esse algoritmo fundamenta-se na comparação iterativa de elementos adjacentes de um vetor, com o objetivo de posicionar progressivamente os maiores valores nas últimas posições da estrutura. Em cada iteração, o algoritmo percorre o vetor comparando pares consecutivos de elementos; sempre que o elemento da esquerda possui valor superior ao da direita, é efetuada a troca (swap) entre eles. Esse processo é repetido até que, ao final de uma passagem completa,

nenhuma troca seja necessária, indicando que o vetor encontra-se totalmente ordenado. Embora o Bubble Sort seja de fácil implementação, ele apresenta baixa eficiência para grandes conjuntos de dados devido ao elevado número de comparações e trocas necessárias. Sua complexidade temporal varia conforme a disposição inicial dos elementos no vetor:

- Melhor caso (vetor já ordenado): o algoritmo realiza apenas $n - 1$ comparações e nenhuma troca, resultando em uma complexidade $O(n)$;
- Pior caso (vetor em ordem inversa): são executadas n passadas, com $n - 1$ comparações em cada uma, totalizando aproximadamente $n^2 - n$ operações, caracterizando uma complexidade $O(n^2)$;
- Caso médio: também apresenta complexidade $O(n^2)$, uma vez que a média entre o melhor e o pior caso resulta em um comportamento quadrático.

Dessa forma, o Bubble Sort é classificado como um algoritmo de ordenação de baixa eficiência, especialmente quando comparado a métodos mais modernos, como Quick Sort, Merge Sort e Heap Sort. Assim, mesmo não sendo indicado para aplicações práticas que envolvam grandes volumes de dados, o Bubble Sort continua sendo amplamente utilizado em ambientes acadêmicos para introduzir o estudo dos algoritmos de ordenação e o raciocínio lógico por trás da organização de dados. (PEDROSO, 2022)

3.3 Selection Sort

O Selection Sort, ou ordenação por seleção cujo funcionamento baseia-se na busca iterativa pelo menor elemento do vetor e em sua colocação na posição correta. Em cada iteração, o algoritmo percorre o segmento não ordenado do vetor para identificar o menor valor, trocando-o com o elemento localizado na posição inicial desse segmento. Esse processo é repetido até que todos os elementos estejam organizados em ordem crescente. O algoritmo pode ser dividido em três etapas principais: definir a posição corrente (i), percorre o vetor a partir de $i + 1$ até o final para encontrar o menor elemento e, por fim, realizar a troca (swap) entre o elemento

de índice i e o menor valor encontrado. Já conforme a W3Schools (2024), essa abordagem garante que, a cada iteração, um novo elemento é colocado de forma definitiva em sua posição correta, reduzindo progressivamente a porção não ordenada do vetor. (SILVA, 2019)

Embora o número de trocas realizadas seja pequeno — no máximo uma por iteração —, o algoritmo executa um número elevado de comparações, o que impacta negativamente sua eficiência em grandes conjuntos de dados. A complexidade temporal do Selection Sort é determinada pelo somatório das comparações feitas a cada passada:

$$T(n) = (n-1) + (n-2) + \dots + 1 = \frac{1}{2}(n^2 - n)$$

Assim, o tempo de execução cresce de forma quadrática, caracterizando uma complexidade $O(n^2)$, tanto no melhor quanto no pior caso. (W3SCHOOLS, s.d.)

3.4 Insertion Sort

O Insertion Sort baseia-se no princípio de inserir, a cada iteração, o próximo elemento da lista em sua posição correta dentro da porção já ordenada do vetor. Assim, após cada nova inserção, a parte inicial do conjunto encontra-se ordenada. Essa lógica se assemelha ao modo como uma pessoa organiza cartas de baralho nas mãos: a cada nova carta retirada, ela é comparada com as demais e colocada no local apropriado, de forma a manter a sequência crescente ou decrescente desejada. O processo de inserção é realizado por meio de comparações sucessivas entre o elemento atual e os anteriores, deslocando-os até que o item seja posicionado corretamente. Por exemplo, na sequência [9, 13, 16, 21, 32, 12], o número 12 é comparado com os anteriores e movido até ocupar sua posição correta, resultando em [9, 12, 13, 16, 21, 32] (SILVA, 2019).

A implementação do Insertion Sort é simples e pode ser feita em diversas linguagens de programação, como Java, C ou Python. Sua estrutura básica utiliza dois laços: o primeiro percorre o vetor a partir do segundo elemento, enquanto o segundo reposiciona o elemento corrente na posição adequada. Trata-se de um algoritmo in-place, pois não requer memória auxiliar significativa além de algumas variáveis temporárias, e estável, visto que mantém a ordem relativa entre elementos de mesmo

valor — característica relevante em contextos de ordenação de registros com múltiplos campos (SILVA,2019).

O desempenho do Insertion Sort está diretamente relacionado à disposição inicial dos elementos na lista. No melhor caso, quando o vetor já se encontra ordenado, o algoritmo realiza apenas uma comparação por interação, apresentando complexidade linear, representada por $O(n)$. Em contrapartida, no pior cenário — quando os elementos estão em ordem completamente inversa — cada novo item precisa ser comparado e deslocado sucessivamente até atingir sua posição correta, resultando em aproximadamente $n^2/2$ operações, o que caracteriza uma complexidade quadrática, $O(n^2)$. Em média, o comportamento do algoritmo também tende a $O(n^2)$, uma vez que, para a maioria das sequências desordenadas, são necessárias múltiplas comparações e movimentações. Apesar disso, o Insertion Sort é considerado eficiente para listas pequenas ou quase ordenadas, além de possuir as vantagens de ser simples, estável e in-place, o que justifica seu uso em determinados contextos de ordenação (SILVA, 2019).

3.5 Quick Sort

O Funcionamento do Quick Sort se fundamenta em um processo essencial denominado particionamento. O particionamento consiste em selecionar um número qualquer presente no array, chamado de pivot, e colocá-lo em uma posição tal que todos os elementos à esquerda são menores ou iguais e todos os elementos à direita são maiores. Segundo Cormen et al. (2009), o Quick Sort é um dos algoritmos de ordenação mais eficientes, pois combina o paradigma de divisão e conquista com uma execução em tempo médio de $O(n \log n)$, sendo amplamente utilizado em implementações práticas. O particionamento é feito da seguinte maneira:

- Array utilizado: [3,8,7,10,0,23,2,1,77,7].
- Antes do particionamento: [3, 8, 7, 10, 0, 23, 2, 1, 77, 7]
- Depois do particionamento: [1, 0, 2, 3, 8, 23, 7, 10, 77, 7]

Observe que todos os elementos à esquerda do pivot são menores ou iguais a ele, enquanto todos os elementos à direita são maiores. Isso não implica que os elementos à esquerda e à direita precisem estar organizados. O conceito de particionamento de Lomuto consiste em identificar os elementos que são menores ou iguais ao pivot e

posicioná-los logo antes dele. Em seguida, no final, coloca-se o pivot à frente de todos eles.

O Quick Sort, então, é a execução de consecutivos particionamentos. Efetua-se o primeiro levando em consideração todo o array ($\text{left} = 0$ e $\text{right} = \text{values.length} - 1$). Depois, leva-se em consideração a esquerda do pivot, ou seja, $\text{left} = 0$ e $\text{right} = \text{index_pivot} - 1$ e a direita do pivot ($\text{left} = \text{index_pivot} + 1$ e $\text{right} = \text{values.length} - 1$). Depois, o mesmo processo é feito com a esquerda e a direita dos novos pivots e assim por diante até que todo o array já tenha sido percorrido ($\text{left} \geq \text{right}$).

3.6 Merge Sort

Merge Sort é um algoritmo eficaz de ordenação baseado na estratégia de divisão e conquista. Se a tarefa é organizar um array comparando seus elementos, do ponto de vista assintótico, $n \cdot \log n$ representa o limite inferior. Em outras palavras, nenhum algoritmo de ordenação por comparação é mais rápido do que $n \cdot \log n$. De maneira formal, todos são $\Omega(n \cdot \log n)$. Uma característica importante do Merge Sort é que sua eficiência é $n \cdot \log n$ para os melhores, piores e casos médios. Em outras palavras, não apenas é $\Omega(n \cdot \log n)$, mas também é $\Theta(n \cdot \log n)$. Isso dá uma garantia de que, independentemente da disposição dos dados em um array, a ordenação será eficiente. O Merge Sort opera com base em uma rotina essencial chamada merge. Conforme Knuth (1998), o Merge Sort é um dos algoritmos mais estáveis e previsíveis, sendo amplamente usado em aplicações que exigem resultados consistentes e comportamento uniforme de desempenho.

O Merge Sort é mais rápido e eficiente do que outros algoritmos de ordenação, como Bubble Sort, Insertion Sort e Selection Sort, especialmente quando se trabalha com grandes volumes de dados. Quando aplicado a pequenos conjuntos de dados, ele não se revela tão eficaz em comparação com outros algoritmos de ordenação. Uma das desvantagens do Merge Sort é o consumo adicional de memória, já que uma cópia do array é criada a cada chamada recursiva da função de ordenação que implementa o algoritmo.

Naturalmente, ele recebe como parâmetro o array a ser processado. Recebe também dois índices: left e right , que determinam os limites em que o algoritmo deve agir. a parte do array que é delimitada por left e middle , onde $\text{middle} = (\text{right} + \text{left}) / 2$ está

ordenada e sabe que a parte do array delimitada por $middle + 1$ e $right$ também está ordenada. Merge Sort é um algoritmo eficiente de ordenação.

Independente do caso (melhor, pior ou médio) o Merge Sort sempre será $n \cdot \log n$. Isso ocorre porque a divisão do problema sempre gera dois sub-problemas com a metade do tamanho do problema original ($2 \cdot T(n/2)$).

O algoritmo fundamenta a ordenação em execuções consecutivas de merge, um procedimento que combina duas seções ordenadas de um array em uma nova seção também ordenada. Embora o Merge Sort esteja na mesma categoria de complexidade do Quick Sort, na prática, ele costuma ser um pouco menos eficiente que o Quick Sort, devido às suas constantes maiores. No entanto, o Merge Sort garante $n \cdot \log n$ para qualquer situação, ao passo que o Quick Sort pode apresentar uma ordenação n^2 no pior cenário, embora isso seja raro.

3.7 Shell Sort

O Shell sort, também conhecido como ordenação Shell, é um algoritmo de ordenação por comparação que melhora o Insertion sort (ordenação por inserção). A principal melhoria é permitir a troca de elementos distantes no início da ordenação, o que diminui consideravelmente a quantidade de movimentações necessárias para ordenar uma lista. Donald Shell apresentou o método em 1959. Segundo Sedgewick e Wayne (2011), o Shell Sort é considerado uma das primeiras tentativas bem-sucedidas de reduzir o tempo de execução de algoritmos quadráticos, alcançando desempenho próximo de $O(n \log n)$ quando bem implementado.

O algoritmo utiliza uma sequência decrescente de intervalos ou "gaps". Uma escolha comum é começar com um gap de $n/2$ (onde n é o número de elementos), depois $n/4$, e assim por diante, até que o gap seja igual a 1. O algoritmo percorre a lista e realiza uma ordenação por inserção para cada sublista definida pelo gap. Para um gap de 4, por exemplo, ele ordenaria as sublistas formadas pelos elementos nas posições (0, 4, 8...), (1, 5, 9...), (2, 6, 10...) e (3, 7, 11...). Após ordenar todas as sublistas para um determinado gap, o valor do gap é reduzido, e o processo se repete. A última passagem sempre usa um gap de 1. Neste ponto, a lista já está parcialmente ordenada, o que torna a ordenação por inserção final muito rápida e eficiente.

A complexidade do Shell sort é bastante variável e depende da sequência de gaps empregada. No pior cenário, apresenta $O(n^2)$ para sequências de gaps mais simples, como a original de Shell. No entanto, no melhor cenário, pode atingir $O(n \log n)$ com sequências de gaps otimizadas, particularmente quando a lista já está quase ordenada.

• **Vantagens:**

1. Mais rápido que o Insertion Sort: Trata-se de uma versão aprimorada do Insertion Sort, especialmente eficaz para conjuntos de dados de tamanho moderado.
2. Ordenação in-place: exige pouca memória extra, o que a torna útil em sistemas com recursos escassos.
3. Adaptável: opera de maneira eficaz com dados que já estão parcialmente organizados.
4. Fácil de implementar: a lógica é uma extensão direta e intuitiva do Insertion Sort.

• **Desvantagens:**

1. Não é estável: não assegura a preservação da ordem original de elementos com valores iguais.
2. Dependência da sequência de gaps: o rendimento é bastante afetado pela seleção da sequência de gaps, podendo ser ineficiente caso a escolha seja inadequada.
3. Desempenho inferior a algoritmos avançados: em conjuntos de dados muito grandes, fica atrás de algoritmos como Merge Sort e Quick Sort.

3.8 Cocktail Shaker Sort

O Cocktail Shaker Sort, também conhecido como Bubble Sort bidirecional ou Shaker Sort, é um algoritmo de ordenação que é uma variação do Bubble Sort. Ele funciona percorrendo a lista em direções alternadas, da esquerda para a direita e depois da direita para a esquerda, para ordenar os elementos. Segundo Knuth (1998), essa variação foi desenvolvida para reduzir a ineficiência do Bubble Sort tradicional, permitindo que o algoritmo percorra a lista em ambas as direções, diminuindo o número de comparações desnecessárias.

O algoritmo opera em duas fases alternadas por passagem, reduzindo o limite do array que está sendo ordenado a cada vez. O algoritmo começa no início do array e move-se para o final, comparando cada par de elementos adjacentes. Se estiverem na ordem errada, ele os troca. Esta passagem "borbulha" o maior elemento não ordenado para sua posição final correta, no final do array, o algoritmo então inverte a direção.

Começando do novo final da parte não ordenada, ele se move para trás em direção ao início, comparando e trocando pares adjacentes. Esta passagem "borbulha" o menor elemento não ordenado para sua posição final correta, no início do array, assim o processo se repete, com cada passagem reduzindo o alcance da parte não ordenada do array. O algoritmo termina quando uma passagem completa (para a frente e para trás) é feita sem nenhuma troca.

3.9 Heap Sort

O Heap Sort é um algoritmo de ordenação que usa uma estrutura especial chamada heap para organizar os dados de forma eficiente. Um heap é uma árvore binária onde cada nó tem no máximo dois filhos e a árvore é completa ou quase completa, ou seja, todos os níveis estão cheios, exceto talvez pelo último, preenchido da esquerda para a direita. (PFITZNER; PINTO; COSTA, 2009.)

“Dado qualquer elemento da árvore, este será sempre maior ou igual que seus filhos diretos.”(PFITZNER; PINTO; COSTA,2009). Isso significa que, em um heap máximo, o valor de cada nó é maior ou igual ao valor de seus filhos. Além do mais, se um nó é maior que seus filhos, e seus filhos são maiores que os netos, então o nó raiz é o maior de todos, garantindo que o maior elemento sempre estará na raiz da árvore.

Segundo Pfitzner, Pinto e Costa (2009) a árvore é implícita, simulada por um vetor e a relação entre pais e filhos é definida por fórmulas matemáticas sendo o pai do nó $i = (i - 1) / 2$; o filho esquerdo $= (i * 2) + 1$; e o filho direito $= (i * 2) + 2$.

A ordenação de um vetor com n elementos utilizando o Heap Sort é feita em dois passos, sendo eles: A transformação do vetor em um heap, ou seja, reorganizar o vetor para que ele satisfaça a propriedade de heap máximo, utilizando o procedimento heapify que, a partir de um nó, ajusta sua posição em relação aos seus filhos, garantindo que o pai seja sempre maior que os filhos. E o segundo passo é a própria ordenação realizando $n-1$ passos, em que cada passo troca o primeiro

elemento com o último do vetor; reduz o tamanho do heap em 1 e chama o heapify no primeiro elemento para restabelecer a propriedade de heap no restante do vetor. (PFITZNER; PINTO; COSTA, 2009.).

Para Pfitzner, Pinto e Costa (2009) “O Heapsort é um excelente método de ordenação, tendo complexidade de pior caso igual a $O(n \log n)$, que é o melhor que se pode obter.” O seu melhor e médio caso também é $O(n \log n)$, pois o Heap Sort não se beneficia de dados já ordenados, isso cria uma vantagem de não usar memória adicional.

3.10 Machine Learning

Machine Learning é uma área de ciência que busca fazer o computador aprender uma tarefa sem ser programada explicitamente. Um modelo de aprendizagem analisa um conjunto de dados brutos para identificar, a partir dos exemplos fornecidos, os padrões necessários para executar sua tarefa. (HOMEM, 2020).

Os algoritmos de Machine Learning normalmente possuem três componentes: processo de decisão, função de erro e processo de atualização. No processo de decisão a máquina estima os padrões nos dados recebidos, a partir de cálculos. Na etapa da função de erro, o computador compara a estimativa encontrada com dados anteriores, caso houver, para avaliar a precisão do cálculo. Já no processo de atualização a máquina analisa o erro e otimiza como o processo de decisão chega à decisão final. (SHARMA, 2021, online).

Existem alguns tipos de aprendizado de máquina que o desenvolvedor irá escolher para treinar. Por exemplo, no aprendizado de máquina supervisionado se tem um conjunto de dados completos com respostas para análise e comparação enquanto se treina uma máquina. Dessa forma, quando o computador recebe um dado novo, ele irá compará-lo com exemplos anteriores para prever a resposta correta. (ZHAO, 2021, online).

Para Homem (2020) muitos problemas podem aparecer durante a abordagem de um modelo de aprendizagem de máquina, mas duas são mais usuais de aparecerem, sendo: problemas de regressão de classificação. Problemas de regressão têm como objetivo prever um valor numérico dentro de um intervalo contínuo, o que significa que a resposta pode ser qualquer número dentro de uma

faixa. Já os problemas de classificação têm como objetivo selecionar um rótulo ou categoria a partir de uma lista pré-definida de opções.

Todos os modelos de ML partem da mesma premissa de que o algoritmo não precisa ser programado com regras de condicional ou repetição. Em vez disso, o computador aprende sozinho os padrões ao ser treinado com muitos dados de exemplo. (HOMEM, 2020).

O aprendizado de máquina não supervisionado acontece quando o modelo de machine learning recebe um conjunto de dados que não possuem rótulos ou respostas corretas. Assim, o computador extrai e analisa as características e a estruturas dos dados. (ZHAO, 2021, online).

Segundo a Google (2019) o aprendizado não supervisionado possui três modelos sendo clustering, regras de associação e redução de dimensionalidade. O método clustering utiliza-se de uma coleta de dados não rotulada e analisa suas características para separá-los em grupos rotulados. No modelo de associação o computador inspeciona pequenas informações de um grande grupo de dados para identificar relações entre si, se utilizando com frequência a função if-else. E no método redução de dimensionalidade o algoritmo descarta as informações desnecessárias e repetidas para melhor visualização dos dados importantes.

No caso do aprendizado de máquina semissupervisionado é o conjunto de dados categorizados e não categorizados. Serve para algoritmos que não possuem um banco de dados completamente rotulados, desse jeito quando há a entrada de dados novos que não possuem uma classe designada, o modelo pode se utilizar do pouco de dados rotulados para atribuir uma resposta adequada. (ZHAO, 2021, online).

Por fim, o aprendizado de máquina por reforço tem como objetivo encontrar o melhor jeito de alcançar um determinado objetivo para ganhar uma recompensa. Quanto melhor o caminho, maior será a recompensa. O algoritmo se baseia a partir de feedbacks anteriores e experiências novas que atribuem um maior retorno. Esse tipo de técnica é muito útil para treinar robôs e suas tomadas de decisões. (ZHAO, 2021, online).

3.11 Deep Learning

Segundo Lecun, Bengio e Hinton(2015) os métodos tradicionais de machine learning não eram capazes de realizarem processamentos de dados mais complexos,

como nos casos de identificação automática das características dos dados dispostos, precisando de um especialista explicar detalhadamente esses atributos para a máquina.

Deep learning tem o propósito de transformar esse processo manual em um processo automático feito pela própria máquina, utilizando-se de vários estágios que identifica e analisa os dados brutos aos poucos até chegar no resultado final, como explicado a seguir:

Uma imagem, por exemplo, chega na forma de uma matriz de valores de pixels. As características aprendidas na primeira camada de representação normalmente detectam a presença ou ausência de bordas em orientações e locais específicos na imagem. A segunda camada normalmente detecta padrões ao identificar arranjos específicos de bordas, independentemente de pequenas variações em suas posições. A terceira camada pode combinar esses padrões em arranjos maiores que correspondem a partes de objetos familiares, e as camadas subsequentes detectariam objetos completos como combinações dessas partes. (LECUN; BENGIO e HINTON, 2015).

Caracterizado por sua capacidade de descobrir estruturas complicadas em dados de alta dimensionalidade, este modelo representou um importante avanço para a IA. Sua eficiência foi comprovada pelo desempenho superior em reconhecimento de imagens e fala, além da geração de resultados promissores em diversas aplicações de processamento de linguagem natural.

Para Ponti e Costa(2017) a diferença entre Machine Learning e Deep Learning é que os métodos do ML buscam diretamente por uma única função que, a partir de um conjunto de parâmetros, transforma os dados de entrada (x) na saída desejada (y). Já os métodos de DL aprendem a função $f(\cdot)$ por meio da composição de múltiplas funções simples, sendo a função final uma cadeia de transformações $f(x) = f_L(\cdots f_2(f_1(x_1))\cdots)$.

Cada função f_l realiza uma transformação nos dados utilizando seu conjunto exclusivo de parâmetros W_l . A rede como um todo é definida pela composição dessas funções: $f_L(\cdots f_2(f_1(x_1, W_1), W_2)\cdots, W_L)$, em que x_1 é a entrada inicial. A saída de uma camada torna-se a entrada da seguinte, formando uma arquitetura de L camadas consecutivas. (PONTI e COSTA, 2017)

3.12 Técnicas de Deep Learning aplicadas à imagem

De acordo com Guarise (2019) “Uma rede neural artificial é um algoritmo computacional bioinspirado no sistema nervoso de animais”, dessa forma esse tipo de estrutura consegue aprender a partir de extensos conjuntos de dados, de modo a conseguir categorizar ou detectar padrões em informações nunca vistas, mas que sejam similares às do treinamento. Deep Learning são sistemas computacionais constituídos por várias camadas dessas redes neurais artificiais.

No entanto, treinar redes neurais comuns pode ser muito difícil e trabalhoso, considerando que todos os neurônios de uma camada se conectam com os demais da próxima camada, criando uma quantidade significativa de parâmetros a serem ajustados. Dessa forma utiliza-se as redes neurais convolucionais, sendo esta técnica amplamente utilizada e fundamental, principalmente para aplicações à imagens. Para ALMEIDA, GOMES, ALMEIDA e BALLARIN (2018):

Nas redes convolucionais cada neurônio da camada de input equivale ao valor de um dos pixels da imagem e, diferentemente do que acontece nas redes neurais comuns, essa camada de input não é achatada, ou seja, não se transforma a matriz de 28x28 pixels em um vetor de 784 valores, mas preservam-se suas dimensões.

Esta rede é estruturada por uma série de estágios, sendo o primeiro estágio composto por dois tipos de camadas: camadas convolucionais e camadas de Pooling. (LECUN; BENGIO e HINTON, 2015). Na camada convolucional, cada neurônio atua como um filtro, representado por uma matriz de pesos de dimensão $k \times k \times d$, onde k define o tamanho espacial do filtro e d sua profundidade. (PONTI e COSTA, 2017).

Cada filtro processa a imagem realizando uma combinação linear dos pixels vizinhos em pequenas regiões, produzindo um mapa de características como saída. Diferente das camadas totalmente conectadas, os pesos aqui são compartilhados espacialmente, o que reduz drasticamente a quantidade de parâmetros. Cada área da imagem analisada por um filtro é denominada campo receptivo local. O mesmo conjunto de pesos do filtro é aplicado a todos esses campos, processando cada pixel de maneira consistente. (PONTI e COSTA, 2017).

Conforme a rede neural processa uma imagem, é comum reduzir as *features maps*, que são “representações geradas por um filtro da camada convolucional” segundo Ponti e Costa(2017). Essa redução é feita por uma operação chamada Pooling, sendo a mais usada o Max Pooling, que seleciona apenas o valor de pixel mais alto.

À medida que a rede fica mais profunda, o número de filtros aumenta, e se o tamanho espacial dos *features maps* se mantiver grande, a quantidade de cálculos fica gigantesca, o que torna o processo lento e pesado. O Pooling, ao reduzir o tamanho dos mapas, diminui drasticamente a quantidade de dados a serem processados pelas camadas seguintes. Dessa forma, a rede passa a analisar a imagem em diferentes escalas, que ajuda a ter uma rede mais robusta. (PONTI e COSTA, 2017).

Para Ponti e Costa (2017), as camadas fully connected (FC), que são camadas tradicionais de redes neurais, estão conectadas a todos os neurônios da camada anterior e cada conexão tem um peso específico. As FC sempre vêm depois das camadas convolucionais, após estas transformarem tensores em vetores. Assim, a FC recebe esses vetores como entrada e os analisa para atuar como classificador final.

A introdução da retropropagação(backpropagation), que é o algoritmo que calcula como os pesos e parâmetros da rede devem ser ajustados para minimizar o erro nas previsões, foi muito importante para o treinamento de redes neurais recorrentes. (LECUN; BENGIO e HINTON, 2015). As RNN são redes que possuem memória, utilizando informações de processamentos anteriores para gerar novas saídas, o que as torna especialmente eficazes para analisar dados em sequência. Esse tipo de rede é muito útil para dados de entradas como vídeo, som e textos. (GUARISE, 2019).

Para Lecun; Bengio e Hinton(2015) “Desde o início dos anos 2000, as ConvNets(redes convolucionais) foram aplicadas com grande sucesso à detecção, segmentação e reconhecimento de objetos e regiões em imagens.”. E este sucesso veio com tudo após o ano de 2012 com o uso eficiente de GPUs, ReLUs, uma nova técnica de regularização chamada dropout, e técnicas para gerar mais exemplos de treinamento aprimorando os existentes. As CNN são atualmente a técnica dominante em reconhecimento, equiparando-se ao desempenho humano em várias tarefas.

4 METODOLOGIA

4.1 Tipo de Pesquisa e Fundamentação Teórica

O tipo de pesquisa utilizado neste trabalho foi Pesquisa bibliográfica, como visto no tópico de fundamentação teórica. Segundo Souza, Oliveira e Alves (2021, p. 65) “A pesquisa bibliográfica está inserida principalmente no meio acadêmico e tem a finalidade de aprimoramento e atualização do conhecimento, através de uma investigação científica de obras já publicadas”. Ou seja, é a partir da pesquisa bibliográfica que é possível mapear o conhecimento existente e construir uma base teórica sólida.

Esse tipo de pesquisa foi de suma importância para esse projeto, pois permitiu o embasamento teórico necessário para compreender o que são algoritmos de ordenação, entender o funcionamento de alguns deles e identificar os métodos de ordenação mais eficientes para grandes volumes de dados espaciais. Também foi eficaz para o entendimento do que é machine e deep learning e o porquê essas ferramentas são necessárias para quase todas as áreas de tecnologia que envolvem análise de dados e reconhecimento de padrões.

Além disso, o tipo de abordagem utilizada será a mista devido a exigência de uma fundamentação teórica robusta para compreensão dos algoritmos, validação experimental das implementações desenvolvidas, de uma análise comparativa de desempenho em cenários reais e da generalização dos resultados para aplicações similares.

4.2 Ambiente de Desenvolvimento e Ferramentas

Todo o código foi desenvolvido em linguagem C devido ao seu controle de baixo nível que permite um manuseio exato de como o programa de comporta e gerencia o acesso direto à memória usando ponteiros. Isso é crucial para implementar algoritmos de ordenação de forma eficiente, já que esses algoritmos envolvem manipulação de índices e trocas e os ponteiros permitem implementações mais eficientes que índices. Além disso, a análise de Complexidade é precisa, pois é possível contar operações individuais, como comparações, trocas e alocações.

Os algoritmos de ordenação foram produzidos em duas ferramentas diferentes sendo as IDE Dev-C++ e Visual Studio Code por causa das vantagens complementares que cada uma oferece. O Dev-C++ proporciona um ambiente simplificado e de rápida compilação para testes iniciais, enquanto o Visual Studio Code oferece ferramentas avançadas de debugging, controle de versão integrado e extensibilidade para desenvolvimento mais complexo. Ademais, utilizar diferentes ambientes de desenvolvimento foi importante para validação de portabilidade do código.

Para utilizar a linguagem C e o programa funcionar corretamente no Visual Studio Code foi necessário instalar um compilador, nesse caso o MiGW, e as extensões "C/C++" da Microsoft e "C/C++ Compile Run" no editor da IDE. Além disso, precisa incluir o MiGW nas variáveis de ambiente para que ele execute o programa.

4.3 Base de Dados

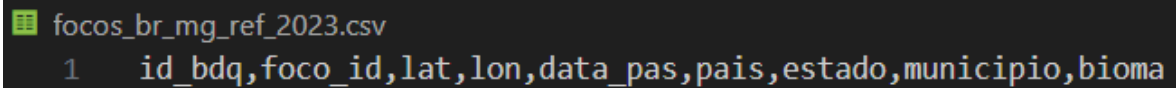
O trabalho foi produzido em cima da base de dados das queimadas ocorridas no estado de Minas Gerais entre os anos de 2023 e 2024 contabilizadas pelo Instituto Nacional de Pesquisas Espaciais(INPE). Os dados possuem 9 campos para sua identificação sendo elas:

1. id_bdq
2. foco_id
3. lat
4. lon
5. data_pas
6. pais
7. estado
8. municipio
9. bioma

O campo id_bdq atua como um identificador único no banco de dados, enquanto foco_id refere-se ao identificador específico do foco de calor detectado. As coordenadas geográficas do evento são registradas nos parâmetros lat (latitude) e lon (longitude). O momento exato da detecção é documentado no campo data_pas, que registra a data e hora do evento. A localização administrativa é definida pelos atributos

país, estado e município, que delimitam a precisão espacial do incidente. Por fim, o atributo bioma classifica o tipo de bioma no qual o foco foi identificado.

Imagem 1 – Parâmetros da base de dados INPE 2023



```
focos_br_mg_ref_2023.csv
1 id_bdq,foco_id,lat,lon,data_pas,país,estado,município,bioma
```

Fonte: Própria (2025)

4.4 Critérios de Ordenação Implementados:

1. Latitude - lat
2. Longitude - lon
3. Data - data_pas
4. Município - município
5. Bioma – bioma

Cada critério foi escolhido devido suas diferentes perspectivas que permitem a análise dos dados, facilitando a identificação de padrões espaciais, temporais e ambientais relevantes para o estudo.

4.5 Implementação dos Algoritmos de Ordenação

Foram implementados oito algoritmos de ordenação representando diferentes paradigmas:

- Bubble Sort ($O(n^2)$) - Base para entendimento
- Insertion Sort ($O(n^2)$) - Eficiente para dados parcialmente ordenados
- Selection Sort ($O(n^2)$) - Simplicidade de implementação
- Quick Sort ($O(n \log n)$ esperado) - Alto desempenho na prática
- Merge Sort ($O(n \log n)$) - Desempenho consistente
- Heap Sort ($O(n \log n)$) - Ordenação in-place com garantia de desempenho
- Shell Sort ($O(n^{3/2})$) - Melhoria do Insertion Sort
- Cocktail Shaker Sort ($O(n^2)$) - Variante bidirecional do Bubble Sort

4.6 Bibliotecas utilizadas

Para o melhor funcionamento do programa foi adicionado cinco bibliotecas e uma diretiva, com cada uma tendo uma função:

- `#define _CRT_SECURE_NO_WARNINGS`: Desativar avisos de segurança do compilador;
- `#include <stdio.h>`: Permite entrada e saída de dados;
- `#include <stdlib.h>`: Possui funções utilitárias gerais;
- `#include <string.h>`: Permite manipulação de strings;
- `#include <stdbool.h>`: Define tipo booleano em C;
- `#include <ctype.h>`: Classificação e conversão de caracteres.

As bibliotecas e diretivas serão incrementadas como mostrado a seguir:

Imagem 2 – Bibliotecas do programa



```
main.c
1  #define _CRT_SECURE_NO_WARNINGS
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5  #include <stdbool.h>
6  #include <ctype.h>
7
8
```

Fonte: Própria (2025)

A arquitetura do código está organizada em módulos, onde cada algoritmo de ordenação é implementado como uma unidade distinta. Na função principal (main), o processo é iniciado com a seleção, por parte do usuário, do arquivo de dados de queimadas a ser processado. Em seguida, é solicitada a escolha do método de ordenação a ser aplicado. Posteriormente, o usuário define o critério de organização dos dados. A implementação de cada algoritmo seguirá uma estrutura padronizada, parecida com a do Bubble Sort, apresentada a seguir:

Imagem 3 – Módulo de Bubble Sort

```

1  #define _CRT_SECURE_NO_WARNINGS
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5  #include <stdbool.h>
6  #include <ctype.h>
7
8  int BubbleSort(char** linhas, void* coluna, int tipo, int n)
9  {
10     int trocas = 0;
11     bool troca = true;
12     while (troca) {
13         int i;
14         troca = false;
15         for (i = 0; i < n - 1; i++) {
16             bool precisa = false;
17             if (tipo == 0 && ((int*)coluna)[i] > ((int*)coluna)[i + 1]) precisa = true;
18             if (tipo == 1 && ((float*)coluna)[i] > ((float*)coluna)[i + 1]) precisa = true;
19             if (tipo == 2 && strcmp(((char**)coluna)[i], ((char**)coluna)[i + 1]) > 0) precisa = true;
20             if (tipo == 3 && ((long long*)coluna)[i] > ((long long*)coluna)[i + 1]) precisa = true;
21             if (precisa) {
22                 if (tipo == 0) { int t = ((int*)coluna)[i]; ((int*)coluna)[i] = ((int*)coluna)[i + 1]; ((int*)coluna)[i + 1] = t; }
23                 if (tipo == 1) { float t = ((float*)coluna)[i]; ((float*)coluna)[i] = ((float*)coluna)[i + 1]; ((float*)coluna)[i + 1] = t; }
24                 if (tipo == 2) { char* t = ((char**)coluna)[i]; ((char**)coluna)[i] = ((char**)coluna)[i + 1]; ((char**)coluna)[i + 1] = t; }
25                 if (tipo == 3) { long long t = ((long long*)coluna)[i]; ((long long*)coluna)[i] = ((long long*)coluna)[i + 1]; ((long long*)coluna)[i + 1] = t; }
26                 trocaLinha(linhas, i, i + 1);
27                 troca = true;
28                 trocas++;
29             }
30         }
31     }
32     return trocas;
}

```

Fonte: Própria (2025)

4.7 Critério de Validação

A validação do programa será realizada mediante a verificação da conformidade entre os resultados gerados e os resultados esperados. O cenário de teste consiste em: após a seleção do arquivo de entrada, do algoritmo de ordenação e do critério de organização, o sistema deve produzir como saída o conjunto de dados ordenado conforme o critério especificado. Adicionalmente, a saída deve incluir métricas de processo, detalhando o número total de registros processados e a quantidade de operações de troca executadas durante a ordenação.

4.8 Estratégia de Testes

Os testes que serão realizados vão ser os testes unitários e os testes integrados. Os testes unitários verificarão o funcionamento isolado das menores partes do código. Dessa forma, o teste ajudará a identificar falhas pequenas que seriam um problema maior futuramente, garantindo que o código funcione

corretamente em sua versão final. Já os testes de integração serão focados em verificar a interação e comunicação entre a base de dados utilizada, no caso os arquivos na INPE, e o programa. Sendo assim, é possível apurar se tanto os dados quanto o código não estão comprometidos.

Os testes unitários serão feitos da seguinte forma:

Imagem 4 – Algoritmo de Ordenação Bubble Sort

```

main.c
25
26 int BubbleSort(char** linhas, void* coluna, int tipo, int n) {
27     int trocas = 0;
28     bool troca = true;
29     while (troca) {
30         int i;
31         troca = false;
32         for (i = 0; i < n - 1; i++) {
33             bool precisa = false;
34             if (tipo == 0 && ((int*)coluna)[i] > ((int*)coluna)[i + 1]) precisa = true;
35             if (tipo == 1 && ((float*)coluna)[i] > ((float*)coluna)[i + 1]) precisa = true;
36             if (tipo == 2 && strcmp(((char**)coluna)[i], ((char**)coluna)[i + 1]) > 0) precisa = true;
37             if (tipo == 3 && ((long long*)coluna)[i] > ((long long*)coluna)[i + 1]) precisa = true;
38             if (precisa) {
39                 if (tipo == 0) { int t = ((int*)coluna)[i]; ((int*)coluna)[i] = ((int*)coluna)[i + 1]; ((int*)coluna)[i + 1] = t; }
40                 if (tipo == 1) { float t = ((float*)coluna)[i]; ((float*)coluna)[i] = ((float*)coluna)[i + 1]; ((float*)coluna)[i + 1] = t; }
41                 if (tipo == 2) { char* t = ((char**)coluna)[i]; ((char**)coluna)[i] = ((char**)coluna)[i + 1]; ((char**)coluna)[i + 1] = t; }
42                 if (tipo == 3) { long long t = ((long long*)coluna)[i]; ((long long*)coluna)[i] = ((long long*)coluna)[i + 1]; ((long long*)coluna)[i + 1] = t; }
43                 trocaLinha(linhas, i, i + 1);
44                 troca = true;
45                 trocas++;
46             }
47         }
48     }
49     return trocas;
50 }
51
52 int main(void) {
53     // Configuração fixa - usando dados de 2023
54     const char* arquivo2023 = "focos_br_mg_ref_2023.csv";
55
56     FILE* arquivo1 = fopen(arquivo2023, "r");

```

Fonte: Própria (2025)

Imagem 5 – Teste Unitário do Bubble Sort

```

1665902584 ,d343810d-0b23-30a4-9fee-ed06f76196e4, -17.904030 , -42.034450 ,2023-12-30 16:39:00,Brasil,MINAS GERAIS,FR
ANCISC|ÓPOLIS,Mata Atl|óntica
1665902587 ,69a9aac2-0dc3-338d-ad42-830edcd87b93, -16.079850 , -40.288930 ,2023-12-30 16:39:00,Brasil,MINAS GERAIS,JA
CINTO,Mata Atl|óntica
1665902588 ,0adbd5ab-b8d5-3e73-900d-15176c2dde67, -16.078300 , -40.277340 ,2023-12-30 16:39:00,Brasil,MINAS GERAIS,JA
CINTO,Mata Atl|óntica
1665902589 ,0657151e-d2bf-3391-bbbb-c42258ba081f, -16.070040 , -40.290210 ,2023-12-30 16:39:00,Brasil,MINAS GERAIS,JA
CINTO,Mata Atl|óntica
1665902590 ,3c16abfe-6b07-360d-984c-3619b8b08beb, -16.068530 , -40.278740 ,2023-12-30 16:39:00,Brasil,MINAS GERAIS,JA
CINTO,Mata Atl|óntica
1665902605 ,6579f7b0-387f-3b55-8410-9cca9ae8837b, -15.834750 , -40.444730 ,2023-12-30 16:39:00,Brasil,MINAS GERAIS,JO
RD|ÊNIA,Mata Atl|óntica
1665902606 ,36d3a881-ca0b-3311-abad-a00ff04436d4, -15.833250 , -40.433460 ,2023-12-30 16:39:00,Brasil,MINAS GERAIS,JO
RD|ÊNIA,Mata Atl|óntica
1665902607 ,54f544b7-3d68-313e-83a5-f2b8c73720a5, -15.807800 , -41.989630 ,2023-12-30 16:39:00,Brasil,MINAS GERAIS,TA
IOBEIRAS,Mata Atl|óntica
1665902608 ,93feaecf-ba30-3027-a30d-23b8f6c684d2, -15.806020 , -41.975270 ,2023-12-30 16:39:00,Brasil,MINAS GERAIS,TA
IOBEIRAS,Mata Atl|óntica
1665902612 ,619c84e1-cbca-302c-b25b-e10fcf01e9b9, -15.400580 , -42.031680 ,2023-12-30 16:39:00,Brasil,MINAS GERAIS,S|
ÃO JO|ÃO DO PARA|ISO,Mata Atl|óntica
1665902616 ,2b0f993f-c5dd-38fb-a23d-03954e03f11f, -15.098320 , -45.431000 ,2023-12-30 16:39:00,Brasil,MINAS GERAIS,JA
NU|ÚRIA,Cerrado

Total de linhas ordenadas: 6499
Total de trocas realizadas: 5528808

-----
Process exited after 3.126 seconds with return value 0
Pressione qualquer tecla para continuar. . . |

```

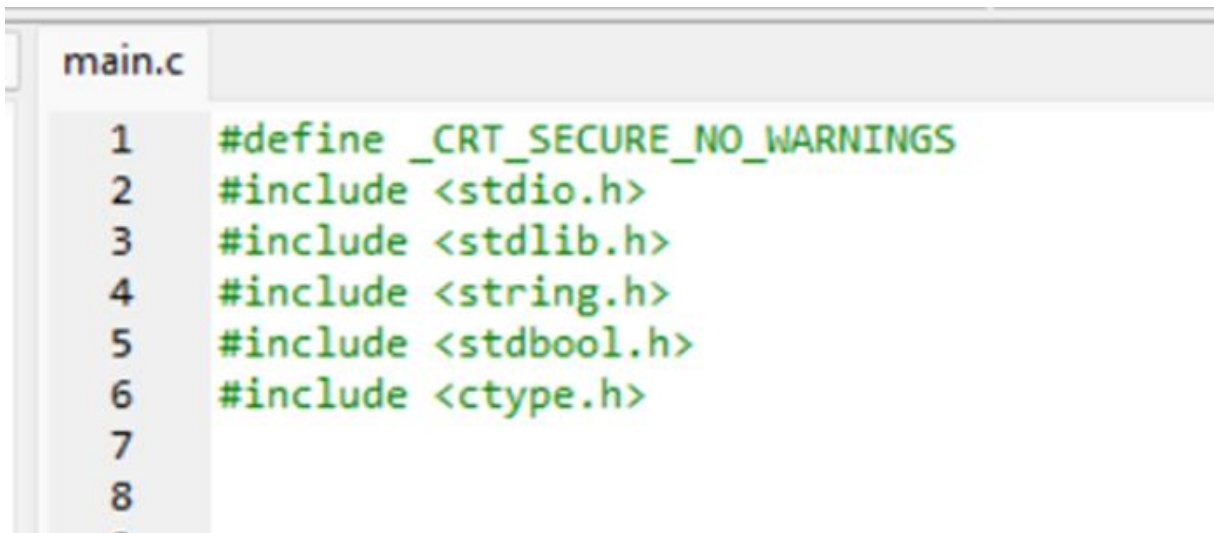
Fonte: Própria (2025)

5 PROJETO/ESTRUTURA DO PROGRAMA

5.1 Bibliotecas

No início do código, é declarado uma diretiva de pré-processamento (`#define _CRT_SECURE_NO_WARNINGS`), utilizada para desativar alertas de segurança gerados pelo compilador da Microsoft em resposta ao uso de funções consideradas menos seguras da biblioteca padrão C. Para o desenvolvimento correto e eficaz do programa, é fundamental o uso de bibliotecas, no código proposto foram utilizadas cinco bibliotecas, sendo elas `"stdio.h"`, `"stdlib.h"`, `"string.h"`, `"stdbool.h"` e `"ctype.h"`. A primeira que foi implementada é a `"<stdio.h>"`, essencial para permitir que o programa interaja com o usuário, dessa maneira ela oferece funcionalidades de entrada e saída de dados, alguns exemplos são as funções: `"printf"`, utilizada para imprimir na tela, `"scanf"`, aplicada para ler a entrada do usuário, `"fopen"` e `"fclose"`, usadas respectivamente para abrir e fechar arquivos. Em seguida foi utilizada a biblioteca `"<stdlib.h>"`, que é responsável por fornecer funções para a conversão de tipos de dados e a manipulação de memória dinâmica, por exemplo: `"atoll"`, `"atoi"` e `"atof"`, usadas para converter strings em respectivamente números do tipo `long long`, `int` e `double`, `"malloc"` e `"free"`, utilizadas para a alocação e liberação de memória. Para permitir o uso de funções para a manipulação de strings, foi implementada a biblioteca `"<string.h>"`, ao decorrer do programa é possível observar o uso de algumas funções desta biblioteca, bem como: `"strcmp"`, que faz a comparação das strings de forma lexicográfica, caractere por caractere, utilizada para colocar as strings em ordem alfabética, e `"strcspn"`, que verifica uma string caractere por caractere e compara se algum caractere dessa primeira string está presente em uma outra string, a função retorna o número de caracteres não encontrados na comparação entre as duas strings e é utilizada para encontrar o índice da primeira ocorrência do caractere comum entre as strings comparadas. A biblioteca `"<stdbool.h>"` é implementada para ser possível utilizar o tipo booleano, os valores `"true"` e `"false"`, pois, por padrão a linguagem C não possui este tipo em sua configuração. Por fim, a última biblioteca incluída é a `"<ctype>"`, que possibilita funções para a manipulação de caracteres, bem como a função `"isdigit"`, que verifica se um caractere ou string contém apenas dígitos numéricos.

Imagem 6 – Declaração da diretiva de pré-pocessamento e das bibliotecas



```

main.c
1  #define _CRT_SECURE_NO_WARNINGS
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5  #include <stdbool.h>
6  #include <ctype.h>
7
8

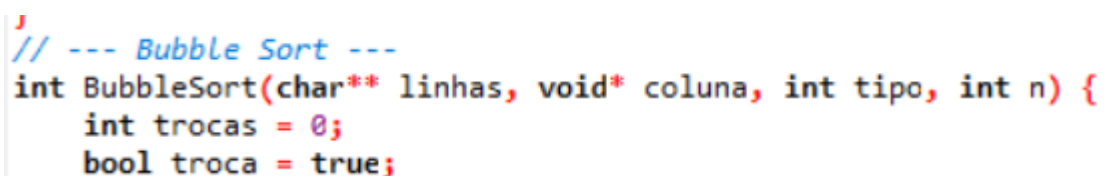
```

Fonte: Própria (2025)

5.2 Bubble Sort

O método de ordenação Bubble sort foi implementado no programa por meio de uma função nomeada de “BubbleSort”, que recebe quatro parâmetros: “linhas” do tipo “char**”, que armazena o endereço de memória de um “char*”, que por sua vez armazena o endereço de memória de um caractere, é utilizado o “char**” para trabalhar com um array de strings, que representam as linhas a serem ordenadas, “coluna” do tipo “void*”, um ponteiro que aponta para a coluna que será ordenada, esse parâmetro pode se referir a quatro tipos de dados (int, float, string e long long), “tipo” do tipo int, um número inteiro que irá indicar o tipo de dado da coluna que será ordenada, 0 para indicar “int”, 1 para “float”, 2 para “string” e 3 para “long long”, e o último parâmetro é denominado “n”, do tipo int, que representa o número de elementos na coluna.

Imagem 7 – Função Bubble Sort e seus parâmetros



```

// --- Bubble Sort ---
int BubbleSort(char** linhas, void* coluna, int tipo, int n) {
    int trocas = 0;
    bool troca = true;

```

Fonte: Própria (2025)

Em seguida, são declaradas duas variáveis, uma do tipo int, chamada de “trocas”, que irá armazenar a quantidade de trocas realizadas durante a ordenação, e por padrão, ela inicia com o valor zero, e a outra do tipo bool, da biblioteca “stdbool.h”, chamada de troca, essa variável indica se houve troca na última iteração do laço, ela inicia com “true”.

Após declarar as variáveis, é iniciado o laço while, que permanecerá em funcionamento enquanto a variável troca for “true”, dessa forma, garantindo que o algoritmo de ordenação permaneça realizando trocas até que o vetor esteja ordenado corretamente. Para o funcionamento do laço interno, é declarada a variável “i”, do tipo int, que não possui valor por padrão, e a variável troca é alterada para false a cada passagem pelo laço, dessa forma assumindo que no início não houve troca. Seguindo o código é possível observar outro laço de repetição, dessa vez um laço for, que será responsável por percorrer do primeiro até o penúltimo elemento da lista, assim o programa irá comparar o elemento com o seu sucessor e efetuar a troca se necessário, e dentro do laço é declarada uma variável nomeada de precisa, do tipo booleana, com o valor inicial “false”, ela é necessária para indicar se a posição atual precisa ou não realizar troca.

Imagem 8 – Laço de repetição principal do Bubble Sort

```
while (troca) {
    int i;
    troca = false;
    for (i = 0; i < n - 1; i++) {
        bool precisa = false;
        if (tipo == 0 && ((int*)coluna)[i] > ((int*)coluna)[i + 1]) precisa = true;
        if (tipo == 1 && ((float*)coluna)[i] > ((float*)coluna)[i + 1]) precisa = true;
        if (tipo == 2 && strcmp(((char**)coluna)[i], ((char**)coluna)[i + 1]) > 0) precisa = true;
        if (tipo == 3 && ((long long*)coluna)[i] > ((long long*)coluna)[i + 1]) precisa = true;
```

Fonte: Própria (2025)

Para distinguir os tipos de dados de maneira mais eficiente, foi implementado uma estrutura condicional, que irá para cada tipo de dado correspondente, converte os elementos para o tipo de dados solicitado, compara os elementos adjacentes, e define a variável “precisa” como “true”, caso o elemento atual for maior que o próximo. Ao “precisa” receber o valor “true”, a troca é necessária, dessa maneira outra estrutura condicional é implementada, dentro deste bloco as condições estão novamente relacionadas ao tipo dos dados e para cada uma a forma de tratamento será de acordo com o seu respectivo tipo de dado, mas em todos os casos, os elementos adjacentes

serão trocados, e para isso se utilizam da variável auxiliar "t", após a estrutura condicional, a função "trocaLinha" é chamada para realizar a troca das linhas correspondentes as colunas que foram trocadas, e depois de uma troca, a variável "troca" será definida como "true", garantindo assim que o laço while continue, já que devem ser feitas outras comparações, além disso, a cada troca a variável "trocas" acrescenta um em seu valor.

Imagem 9 - Estrutura condicional para verificar se a troca é necessária

```

if (precisa) {
    if (tipo == 0) { int t = ((int*)coluna)[i]; ((int*)coluna)[i] = ((int*)coluna)[i + 1]; ((int*)coluna)[i + 1] = t; }
    if (tipo == 1) { float t = ((float*)coluna)[i]; ((float*)coluna)[i] = ((float*)coluna)[i + 1]; ((float*)coluna)[i + 1] = t; }
    if (tipo == 2) { char* t = ((char**)coluna)[i]; ((char**)coluna)[i] = ((char**)coluna)[i + 1]; ((char**)coluna)[i + 1] = t; }
    if (tipo == 3) { long long t = ((long long*)coluna)[i]; ((long long*)coluna)[i] = ((long long*)coluna)[i + 1]; ((long long*)coluna)[i + 1] = t; }
    trocaLinha(linhas, i, i + 1);
    troca = true;
    trocas++;
}
}
return trocas;
}

```

Fonte: Própria (2025)

O laço for irá iterar até que todos os elementos sejam percorridos, e o laço while se repete até que todos os elementos estejam ordenados corretamente. Após terminar o laço while, ou seja, quando todos os elementos estiverem em suas posições corretas, a função retorna o número de trocas realizadas para a ordenação.

5.2 Cocktail Shaker Sort

O método de ordenação Cocktail Shaker Sort foi implementado no programa por meio de um função nomeada de "CocktailShakerSort", a sua estrutura é muito parecida com a do Bubble Sort, mas se diferencia por ser bidirecional, ou seja, a ordenação é feita da esquerda para a direita e depois da direita para a esquerda, ela recebe os mesmos quatro parâmetros da função "BubbleSort", sendo eles: "linhas" do tipo "char**", que armazena o endereço de memória de um "char*", que por sua vez armazena o endereço de memória de um caractere, é utilizado o "char**" para trabalhar com um array de strings, que representam as linhas a serem ordenadas, "coluna" do tipo "void*", um ponteiro que aponta para a coluna que será ordenada, esse parâmetro pode se referir a quatro tipos de dados (int, float, string e long long), "tipo" do tipo int, um número inteiro que irá indicar o tipo de dado da coluna que será ordenada, 0 para indicar "int, 1 para "float", 2 para "string" e 3 para "long long", e o

último parâmetro é denominado “n”, do tipo int, que representa o número de elementos na coluna.

Imagem 10 – Função do Cocktail Shaker Sort e seus parâmetros

```
// --- Cocktail Shaker Sort ---
int CocktailShakerSort(char** linhas, void* coluna, int tipo, int n) {
    int trocas = 0;
    bool troca = true;
    int inicio = 0, fim = n - 1;
```

Fonte: Própria (2025)

Em seguida, são declaradas quatro variáveis, uma do tipo int, chamada de “trocas”, que irá armazenar a quantidade de trocas realizadas durante a ordenação, ela inicia com o valor zero, uma outra do tipo bool, da biblioteca “stdbool.h”, chamada de “troca”, essa variável indica se houve troca na última iteração do laço, ela inicia com “true”, e as duas variáveis responsáveis por permitir o bidirecionamento desse método de ordenação, já que delimitam o início e o fim do intervalo da lista ordenada, sendo elas “inicio” do tipo int, que possui valor zero, e “fim”, do tipo int e possui o valor n-1.

Após declarar as variáveis, é iniciado o laço while, que permanecerá em loop enquanto houver trocas a serem feitas, assim, garantindo que o algoritmo de ordenação permaneça realizando trocas até que o vetor esteja ordenado corretamente.

Para o funcionamento do laço interno, é declarada a variável “i”, do tipo int, que não possui valor por padrão, e a variável troca é alterada para false a cada passagem pelo laço, dessa forma assumindo que no início não houve troca.

Seguindo o código é possível observar outro laço de repetição, dessa vez um laço for, que será responsável por percorrer do início até o fim da lista, dessa maneira, da esquerda para a direita, assim o programa irá comparar o elemento com o seu sucessor e efetuar a troca se necessário, e dentro do laço é declarada uma variável nomeada de precisa, do tipo booleana, com o valor inicial “false”, ela é necessária para indicar se a posição atual precisa ou não realizar troca.

Imagem 11 – Laço de repetição que ordena da esquerda para a direita

```

while (troca) {
    int i;
    troca = false;
    for (i = inicio; i < fim; i++) {
        bool precisa = false;
        if (tipo == 0 && ((int*)coluna)[i] > ((int*)coluna)[i + 1]) precisa = true;
        if (tipo == 1 && ((float*)coluna)[i] > ((float*)coluna)[i + 1]) precisa = true;
        if (tipo == 2 && strcmp(((char**)coluna)[i], ((char**)coluna)[i + 1]) > 0) precisa = true;
        if (tipo == 3 && ((long long*)coluna)[i] > ((long long*)coluna)[i + 1]) precisa = true;
    }
}

```

Fonte: Própria (2025)

Para distinguir os tipos de dados de maneira mais eficiente, foi implementado uma estrutura condicional, que irá para cada tipo de dado correspondente, converte os elementos para o tipo de dados solicitado, compara os elementos adjacentes, e define a variável “precisa” como “true”, caso o elemento atual for maior que o próximo. Ao “precisa” receber o valor “true”, a troca é necessária, dessa maneira outra estrutura condicional é implementada, dentro deste bloco as condições estão novamente relacionadas ao tipo dos dados e para cada uma a forma de tratamento será de acordo com o seu respectivo tipo de dado, mas em todos os casos, os elementos adjacentes serão trocados, e para isso se utilizam da variável auxiliar “t”, após a estrutura condicional, a função “trocaLinha” é chamada para realizar a troca das linhas correspondentes as colunas que foram trocadas, e depois de uma troca, a variável “troca” será definida como “true”, garantindo assim que o laço while continue, já que devem ser feitas outras comparações, além disso, a cada troca a variável “trocas” acrescenta um em seu valor.

Imagem 12 – Estrutura condicional para verificar se a troca é necessária

```

if (precisa) {
    if (tipo == 0) { int t = ((int*)coluna)[i]; ((int*)coluna)[i] = ((int*)coluna)[i + 1]; ((int*)coluna)[i + 1] = t; }
    if (tipo == 1) { float t = ((float*)coluna)[i]; ((float*)coluna)[i] = ((float*)coluna)[i + 1]; ((float*)coluna)[i + 1] = t; }
    if (tipo == 2) { char* t = ((char**)coluna)[i]; ((char**)coluna)[i] = ((char**)coluna)[i + 1]; ((char**)coluna)[i + 1] = t; }
    if (tipo == 3) { long long t = ((long long*)coluna)[i]; ((long long*)coluna)[i] = ((long long*)coluna)[i + 1]; ((long long*)coluna)[i + 1] = t; }
    trocaLinha(linhas, i, i + 1);
    troca = true;
    trocas++;
}

```

Fonte: Própria (2025)

Se ao percorrer da esquerda para a direita, dentro do primeiro passo, e não for efetuada nenhuma troca, o laço é encerrado, por meio do comando “break, caso contrário, a variável “troca” é resetada para false e a variável “fim” é decrescida em um, já que após a primeira varredura o último elemento do vetor está em sua posição correta, dessa maneira, a comparação irá começar a partir do penúltimo elemento.

Imagem 13 – Estrutura condicional que encerra o laço caso não houve trocas

```
if (!troca) break;
troca = false;
fim--;
```

Fonte: Própria (2025)

Seguindo o código é possível observar outro laço de repetição, um laço for, que será responsável por percorrer do final até o início da lista, dessa maneira, da direita para a esquerda, assim o programa irá comparar o elemento com o seu antecessor e efetuar a troca se necessário, e dentro do laço é declarada uma variável nomeada de *precisa*, do tipo booleana, com o valor inicial “false”, ela é necessária para indicar se a posição atual precisa ou não realizar troca.

Imagem 14 – Laço de repetição que ordena da direita para a esquerda

```
for (i = fim; i > inicio; i--) {
    bool precisa = false;
    if (tipo == 0 && ((int*)coluna)[i - 1] > ((int*)coluna)[i]) precisa = true;
    if (tipo == 1 && ((float*)coluna)[i - 1] > ((float*)coluna)[i]) precisa = true;
    if (tipo == 2 && strcmp(((char**)coluna)[i - 1], ((char**)coluna)[i]) > 0) precisa = true;
    if (tipo == 3 && ((long long*)coluna)[i - 1] > ((long long*)coluna)[i]) precisa = true;
```

Fonte: Própria (2025)

Para distinguir os tipos de dados de maneira mais eficiente, foi implementado uma estrutura condicional, que irá para cada tipo de dado correspondente, converte os elementos para o tipo de dados solicitado, compara os elementos adjacentes, e define a variável “*precisa*” como “true”, caso o elemento anterior for maior que o atual. Ao “*precisa*” receber o valor “true”, a troca é necessária, dessa maneira outra estrutura condicional é implementada, dentro deste bloco as condições estão novamente relacionadas ao tipo dos dados e para cada uma a forma de tratamento será de acordo com o seu respectivo tipo de dado, mas em todos os casos, os elementos adjacentes serão trocados, e para isso se utilizam da variável auxiliar “*t*”, após a estrutura condicional, a função “*trocaLinha*” é chamada para realizar a troca das linhas correspondentes as colunas que foram trocadas, e depois de uma troca, a variável “*troca*” será definida como “true”, garantindo assim que o laço while continue, já que devem ser feitas outras comparações, além disso, a cada troca a variável “*trocadas*” acrescenta um em seu valor. Após o segundo passo de ordenação, da direita para a esquerda, a variável “*inicio*” é acrescida em um, assim novamente restringindo a área

de ordenação, já que após este processo, o primeiro elemento do vetor está corretamente ordenado.

Imagem 15 – Estrutura condicional para verificar se a troca é necessária

```

if (precisa) {
    if (tipo == 0) { int t = ((int*)coluna)[i - 1]; ((int*)coluna)[i - 1] = ((int*)coluna)[i]; ((int*)coluna)[i] = t; }
    if (tipo == 1) { float t = ((float*)coluna)[i - 1]; ((float*)coluna)[i - 1] = ((float*)coluna)[i]; ((float*)coluna)[i] = t; }
    if (tipo == 2) { char* t = ((char**)coluna)[i - 1]; ((char**)coluna)[i - 1] = ((char**)coluna)[i]; ((char**)coluna)[i] = t; }
    if (tipo == 3) { long long t = ((long long*)coluna)[i - 1]; ((long long*)coluna)[i - 1] = ((long long*)coluna)[i]; ((long long*)coluna)[i] = t; }
    trocalinha(linhas, i - 1, i);
    troca = true;
    trocas++;
}
}
inicio++;
}
return trocas;
}

```

Fonte: Própria (2025)

O laço for irá iterar até que todos os elementos sejam percorridos, e o laço while se repete até que todos os elementos estejam ordenados corretamente. Após terminar o laço while, ou seja, quando todos os elementos estiverem em suas posições corretas, a função retorna o número de trocas realizadas para a ordenação.

5.3 Insertion Sort

O algoritmo de ordenação Insertion Sort foi implementado no programa por meio de uma função chamada “InsertionSort”, a qual recebe quatro parâmetros. O primeiro é “linhas”, do tipo “char**”, que armazena o endereço de memória de um ponteiro do tipo “char*”, o qual, por sua vez, aponta para o endereço de memória de um caractere. Esse tipo de declaração “char**” é utilizado para manipular um array de strings, que representa as linhas que serão ordenadas. O segundo parâmetro é “coluna”, declarado como “void*”, um ponteiro genérico que aponta para a coluna a ser ordenada. Esse ponteiro pode se referir a quatro tipos diferentes de dados: int, float, string e long long. O terceiro parâmetro é “tipo”, do tipo int, responsável por indicar qual é o tipo de dado da coluna que está sendo ordenada, sendo 0 para int, 1 para float, 2 para string e 3 para long long. Por fim, o último parâmetro é “n”, também do tipo int, que representa a quantidade de elementos existentes na coluna.

Imagem 16 – Função do InsertionSort e seus parâmetros

```
// --- Insertion Sort ---
int InsertionSort(char** linhas, void* coluna, int tipo, int n) {
    int trocas = 0;
    int i;
```

Fonte: Própria (2025)

Dentro da função, são declaradas duas variáveis do tipo int, a primeira denominada “trocas”, que é utilizada para armazenar a quantidade de trocas realizadas durante a ordenação do vetor, ela possui valor inicial de zero, e a segunda é chamada “i”, que representa o índice usado para percorrer o vetor dentro do laço de repetição, não possui valor inicial.

Após a declaração das variáveis, o laço de repetição principal do programa, “for”, é inicializado, e ele começa definindo o valor de i para 1, já que a ordenação deverá ser inicializada no segundo elemento da lista, e não no primeiro elemento da lista, e percorrerá a lista até o final, dessa forma, a cada iteração, o programa irá inserir o elemento [i] em sua respectiva posição dentro da parte que já está ordenada, que é à esquerda. Em seguida, são declaradas variáveis auxiliares, uma do tipo int chamada “j”, que define o índice do elemento anterior, por isso recebe o valor de i – 1, e uma outra do tipo char* denominada de linhaChave, que serve para armazenar a linha correspondente ao elemento que está sendo ordenado, por isso seu valor é linhas[i].

Para tratar com os diferentes tipos de dados, foi implementado uma estrutura condicional, que para cada tipo irá criar uma variável chamada “chave”, do tipo que está sendo requisitado e irá guardar o valor da posição “i”. Em seguida, o loop interno while é acionado, ele tem o papel fundamental para a realocação dos elementos na lista, e move todos os elementos maiores que a chave, para uma posição a frente, e permanecerá iterando enquanto j for maior ou igual a 0, ou seja, se há elementos anterior e o elemento da coluna[j] for maior do que a chave, ao atender estes requisitos, os elementos são movidos da posição j irá para a posição j+1, bem como suas respectivas linhas, garantindo assim a integridade dos dados, irá mover o índice para uma posição a esquerda, j--, para comparar a chave com o outro elemento anterior e incrementa um na variável “trocas”.

Imagem 17 – Laço de repetição principal do algoritmo

```

for (i = 1; i < n; i++) {
    int j = i - 1;
    char* linhaChave = linhas[i];
    if (tipo == 0) {
        int chave = ((int*)coluna)[i];
        while (j >= 0 && ((int*)coluna)[j] > chave) {
            ((int*)coluna)[j + 1] = ((int*)coluna)[j];
            linhas[j + 1] = linhas[j];
            j--;
            trocas++;
        }
        ((int*)coluna)[j + 1] = chave;
    }
}

```

Fonte: Própria (2025)

Após encontrar a posição certa, o valor da variável “chave” é inserido na posição j+1, pois foi encontrado um elemento menor ou igual ao valor da chave, assim ao final do laço for, a lista se encontra devidamente ordenada e a função retorna a quantidade de operações realizadas para a ordenação correta.

Imagem 18 – Inserção do elemento chave na posição correta

```

    }
    linhas[j + 1] = linhaChave;
}
return trocas;
}

```

Fonte: Própria (2025)

5.4 Shell Sort

O algoritmo Shell Sort é uma variação melhorada do Insertion Sort, essa versão foi desenvolvida visando diminuir o número de comparações e movimentações necessárias para ordenar grandes quantidades de dados. No programa, essa lógica está implementada dentro da função “ShellSort”, que recebe como parâmetros de linha completas(“linhas”), o vetor com os valores da coluna escolhida(“coluna”), o tipo de dado a ser ordenado(“tipo”) e total de elemento a ser ordenados(“n”).

Imagem 19 – Função ShellSort

```

int ShellSort(char** linhas, void* coluna, int tipo, int n) {

```

Fonte: Própria (2025)

O funcionamento do algoritmo depende do uso de um intervalo de comparação, denominado “gap”, que, a princípio, é estabelecido como a metade do total de elementos ($n / 2$), e vai sendo reduzido pela metade novamente a cada iteração por meio do laço “while”. Esse intervalo possibilita a comparação de elementos afastados, antecipando a correção de grandes movimentações que ocorreriam em um “Insertion Sort” convencional. A cada iteração, o “gap” é dividido por dois, até o valor chegar em 1, momento em que o vetor se torna totalmente ordenado.

Imagem 20 – Declarando variáveis e laço “while” para divisão do gap

```
int trocas = 0;
int i, j, gap;

gap = n / 2;

while (gap > 0) {
    for (i = gap; i < n; i++) {
        bool trocou = false;
```

Fonte: Própria (2025)

Dentro do laço principal, o programa percorre o vetor a partir do índice correspondente ao valor do “gap” até o final, realizando comparações e trocas de acordo com o tipo de dado estabelecido. A estrutura condicional “if” determina o tipo e aplica o procedimento correspondente: números inteiros são considerados “int”, números reais são considerados “float”, textos são considerados “char*” e datas são consideradas “long long”. Para cada elemento, o valor atual é armazenado em uma variável temporária (“temp”), e um laço interno desloca os valores maiores que a variável “temp” em intervalos iguais a “gap”. Isso gera “lacunas” no vetor, nas quais o valor temporário é colocado na posição adequada. Sempre que ocorre uma troca, a função “trocaLinha()” é invocada sempre que uma troca acontece, a fim de manter a sincronização das linhas completas com os dados da coluna, e o contador “trocas” é aumentado.

Imagem 21 – Laço principal da função “ShellSort” para valores “int”

```

if (tipo == 0) {
    int temp = ((int*)coluna)[i];
    j = i;
    while (j >= gap && ((int*)coluna)[j - gap] > temp) {
        ((int*)coluna)[j] = ((int*)coluna)[j - gap];
        trocaLinha(linhas, j, j - gap);
        j -= gap;
        trocas++;
        trocou = true;
    }
    ((int*)coluna)[j] = temp;
}

```

Fonte: Própria (2025)

5.5 Heap Sort

O algoritmo “Heap Sort” é baseado em uma estrutura de dados chamada heap, que pode ser representada como uma árvore binária completa, onde cada nó pai tem um valor maior (ou menor, dependendo da ordem desejada) do que seus filhos. No código, essa lógica é separada em duas funções: “HeapSort”, responsável pela execução completa do algoritmo, e heapify, o qual, sempre que necessário, realiza um ajuste local da estrutura de “heap”.

A função “heapify” recebe os parâmetros de linha completas (“linhas”), o vetor com os valores da coluna escolhida (“coluna”), o tipo de dado a ser ordenado (“tipo”) e total de elemento a ser ordenados (“n”) e um índice (“i”) da função “HeapSort”, e assegura que uma subárvore com raiz no índice “i” cumpra a propriedade do heap máximo, isto é, o valor guardado no nó pai deve ser maior que os valores dos seus filhos.

Imagem 22 – Função heapify

```

int heapify(char** linhas, void* coluna, int tipo, int n, int i) {

```

Fonte: Própria (2025)

Primeiramente, ela estabelece as variáveis “maior = i”, “esq = 2 * i + 1” e “dir = 2 * i + 2”, que indicam o índice do elemento atual e os índices de seus filhos à esquerda e à direita, respectivamente. Em seguida, o programa verifica o tipo de dado armazenado no vetor “coluna”, e faz as comparações necessárias para verificar se algum dos filhos tem um valor maior que o nó pai. Se isso acontecer, o índice “maior” é atualizado para referenciar o filho que possui o maior valor.

Imagem 23 – Variáveis para definir posição dos filhos no heap

```
int trocas = 0;
int maior = i;
int esq = 2 * i + 1;
int dir = 2 * i + 2;
```

Fonte: Própria (2025)

Quando o índice “maior” é diferente de “i”, o programa realiza a troca de elementos garantindo que o maior valor suba para a posição do nó pai. Essa troca é realizada tanto no vetor “linha”, quanto no vetor “coluna”, utilizando a função “trocaLinha(linhas, i, maior)” para preservar a correspondência com os dados originais. Sucessivamente, a função é chamada recursivamente (“heapify(linhas, coluna, tipo, n, maior)”) para manter a propriedade do heap seja mantida também nas subárvores afetadas.

Imagem 24 – Laço principal da função heapify para int

```
if (tipo == 0) {
    if (esq < n && ((int*)coluna)[esq] > ((int*)coluna)[maior]) maior = esq;
    if (dir < n && ((int*)coluna)[dir] > ((int*)coluna)[maior]) maior = dir;
    if (maior != i) {
        int t = ((int*)coluna)[i];
        ((int*)coluna)[i] = ((int*)coluna)[maior];
        ((int*)coluna)[maior] = t;
        trocaLinha(linhas, i, maior);
        trocas++;
        trocas += heapify(linhas, coluna, tipo, n, maior);
    }
}
```

Fonte: Própria (2025)

A função “HeapSort” dá início ao processo recebendo os parâmetros de linha completas (“linhas”), o vetor com os valores da coluna escolhida (“coluna”), o tipo de dado a ser ordenado (“tipo”) e total de elemento a ser ordenados (“n”) definidos na função “main()”. Depois da recepção dos parâmetros, coordena o algoritmo em duas etapas principais: a construção inicial do heap *máximo* e a ordenação propriamente dita. Na primeira fase, o laço “for ($i = n / 2 - 1$; $i \geq 0$; $i--$)” chama a função “heapify” de forma decrescente, assegurando que todos os nós não-folha constituam um “heap” válido.

Imagem 25 – Função HeapSort e laço que garante a formação de um heap válido

```
int HeapSort(char** linhas, void* coluna, int tipo, int n) {
    int trocas = 0;
    int i;
    for (i = n / 2 - 1; i >= 0; i--)
        trocas += heapify(linhas, coluna, tipo, n, i);
}
```

Fonte: Própria (2025)

Na segunda fase, o código percorre o vetor na ordem inversa, substituindo o primeiro elemento (o maior valor do “heap”) pelo último elemento que ainda não foi ordenado. Nessa operação o maior vetor do heap é removido e é posicionado na parte final do vetor, diminuindo o tamanho lógico do heap e chamando novamente o “heapify” para reorganizar a estrutura restante.

Imagem 26 – Laço principal da função HeapSort

```
for (i = n - 1; i > 0; i--) {
    if (tipo == 0) { int t = ((int*)coluna)[0]; ((int*)coluna)[0] = ((int*)coluna)[i]; ((int*)coluna)[i] = t; }
    if (tipo == 1) { float t = ((float*)coluna)[0]; ((float*)coluna)[0] = ((float*)coluna)[i]; ((float*)coluna)[i] = t; }
    if (tipo == 2) { char* t = ((char**)coluna)[0]; ((char**)coluna)[0] = ((char**)coluna)[i]; ((char**)coluna)[i] = t; }
    if (tipo == 3) { long long t = ((long long*)coluna)[0]; ((long long*)coluna)[0] = ((long long*)coluna)[i]; ((long long*)coluna)[i] = t; }
    trocaLinha(linhas, 0, i);
    trocas++;
    trocas += heapify(linhas, coluna, tipo, i, 0);
}
return trocas;
```

Fonte: Própria (2025)

Esse procedimento continua até que todos os elementos estejam organizados em ordem crescente. A cada iteração, os vetores “coluna” e “linhas” são atualizados de forma sincronizada, mantendo a conexão entre os dados originais. Ao final da execução, a função retorna o contador de trocas acumuladas, que serve como uma medida do custo em movimentações.

5.6 Selection Sort

A função SelectionSort foi implementada no programa através da função nomeada de “SelectionSort” que recebe quatro parâmetros. O primeiro parâmetro é “linhas”, do tipo “char*”, que deve ser entendido como um vetor de “strings” e que contém os endereços de memória das linhas de dados que precisam ser ordenadas. O segundo parâmetro é “colunas”, do tipo “void*”, um ponteiro genérico que indica a

coluna de dados para ser ordenada. Esse parâmetro pode representar quatro tipos de dados diferentes, sejam eles string, int, float e long long. O terceiro parâmetro é “tipo”, do tipo int, que representa um número inteiro que irá indicar o tipo de dados da coluna: 0 para “int”, 1 para “float”, 2 para “string” e 3 para “long long”. O último parâmetro “n” do tipo “int”, representa a quantidade total de elementos na coluna.

Imagem 27 - Função Selection Sort

```
int SelectionSort(char** linhas, void* coluna, int tipo, int n)
```

Fonte: Própria (2025)

Em seguida, é declarada a variável "trocas", do tipo “int”, que irá armazenar a quantidade total de trocas realizadas ao longo do processo de ordenação, iniciando com o valor zero. O algoritmo inicia seu funcionamento com o laço externo (for (i = 0; i < n; i++)), que é responsável por percorrer todo o vetor, definindo a posição atual (i) onde o próximo menor elemento deve ser colocado. Dentro desse laço, declara-se a variável min, do tipo “int”, que armazena o valor de “i” com o objetivo de guardar o índice do menor elemento presente na sub-lista que ainda não foi ordenada.

Imagem 28 – Inicializa o contador de trocas

```
int trocas = 0;
int i,j;
for (i = 0; i < n; i++) {
    int min = i;
```

Fonte: Própria (2025)

Para achar esse menor elemento, é iniciado o laço interno (for(j = i + 1; j < n; j++)), que é o responsável por percorrer a sub-lista a partir do elemento seguinte de “i” até o final do vetor. Nesse laço, a variável booleana menor é declarada e inicializada como falsa, funcionando como um sinalizador de comparação. Para distinguir e tratar os dados de maneira eficiente, é implementada uma estrutura condicional que, para cada tipo de dado correspondente, converte os elementos para o tipo de dado solicitado, compara o elemento atual do laço interno (coluna[j]) com o menor elemento registrado até o momento (coluna[min]), e define a variável menor como “true” caso o

elemento em “j” seja estritamente menor. Se a variável menor for definida como “true”, o índice “min” é atualizado para j, registrando o novo índice do menor elemento.

Imagem 29 - Laço interno do Selection Sort

```
for (j = i + 1; j < n; j++) {
    bool menor = false;
    if (tipo == 0 && ((int*)coluna)[j] < ((int*)coluna)[min]) menor = true;
    if (tipo == 1 && ((float*)coluna)[j] < ((float*)coluna)[min]) menor = true;
    if (tipo == 2 && strcmp(((char**)coluna)[j], ((char**)coluna)[min]) < 0) menor = true;
    if (tipo == 3 && ((long long*)coluna)[j] < ((long long*)coluna)[min]) menor = true;
    if (menor) min = j;
}
```

Fonte: Própria (2025)

Após a conclusão do laço interno, a variável min conterá o índice do elemento que é o menor em toda a sub-lista não ordenada. Uma estrutura condicional verifica então se o índice do mínimo (min) é diferente do índice atual (i). Se for diferente, significa que uma troca precisa ser feita. Nesse bloco, outras estruturas condicionais são usadas para lidar com a troca conforme o tipo de dado, utilizando uma variável auxiliar “t” para efetuar a permutação dos valores entre “coluna[i]” e “coluna[min]”. A função auxiliar “trocaLinha” é então chamada, usando os índices “i” e “min” para garantir que a linha de dados inteira relacionada ao valor trocado também se desloque, preservando a coerência. Por fim, o contador “trocas” retorna o número de trocas efetuadas

Imagem 30 – Estrutura condicional

```
if (min != i) {
    if (tipo == 0) { int t = ((int*)coluna)[i]; ((int*)coluna)[i] = ((int*)coluna)[min]; ((int*)coluna)[min] = t; }
    if (tipo == 1) { float t = ((float*)coluna)[i]; ((float*)coluna)[i] = ((float*)coluna)[min]; ((float*)coluna)[min] = t; }
    if (tipo == 2) { char* t = ((char**)coluna)[i]; ((char**)coluna)[i] = ((char**)coluna)[min]; ((char**)coluna)[min] = t; }
    if (tipo == 3) { long long t = ((long long*)coluna)[i]; ((long long*)coluna)[i] = ((long long*)coluna)[min]; ((long long*)coluna)[min] = t; }
    trocaLinha(linhas, i, min);
    trocas++;
}
return trocas;
```

Fonte: Própria (2025)

5.7 Quick Sort

A implementação do método de ordenação QuickSort foi responsável por particionar o vetor de dados “coluna” e sincronizar com as “linhas”. A função começa declarando e inicializando a variável “trocas” como zero, que acumulará o número de

movimentações realizadas no particionamento atual e nas chamadas recursivas subsequentes.

Imagem 31 - Função Quick Sort

```
int QuickSortRec(char** linhas, void* coluna, int tipo, int inicio, int fim) {
    int trocas = 0;
```

Fonte: Própria (2025)

A linha seguinte contém a condição de parada da recursão: “if (inicio >= fim) return trocas”;. Se o índice inicial for maior ou igual ao final, o segmento está ordenado (possui zero ou um elemento), e a função retorna. Em seguida, o pivô é definido como o primeiro elemento do segmento (pivoIndex = inicio), e dois ponteiros são inicializados: i começa logo após o pivô (inicio + 1), movendo-se para a direita, e j começa no final do segmento (fim), movendo-se para a esquerda.

Imagem 32 - Condição de recursão

```
if (inicio >= fim) return trocas;

int pivoIndex = inicio;
int i = inicio + 1;
int j = fim;
```

Fonte: Própria (2025)

O particionamento é executado dentro do laço “while” (i <= j). Em cada iteração, as variáveis booleanas “condicaoEsquerda” e “condicaoDireita” são reinicializadas como “false”.

Imagem 33 – Laço While

```
while (i <= j) {
    bool condicaoEsquerda = false;
    bool condicaoDireita = false;
```

Fonte: Própria (2025)

Uma série de estruturas “if” encadeadas verificam o ponteiro “i”, se o elemento atual é menor ou igual ao pivô, e outra série de “if encadeada verifica para o ponteiro

“j”, se o elemento atual é maior que o pivô. Essas verificações são vitais: elas usam o parâmetro tipo para realizar o casting correto ((int*) coluna, (float*) coluna, etc.) e utilizam “strcmp” para comparações de “strings”. Se a “condicaoEsquerda” for verdadeira, i avança (i++) pois o elemento está no lugar correto (isso ocorre dentro de um “if”). Caso contrário, se a “condicaoDireita” for verdadeira, j recua (j--).

Imagem 34 – Estruturas encadeadas if

```
if (tipo == 0 && ((int*)coluna)[i] <= ((int*)coluna)[pivoIndex]) condicaoEsquerda = true;
if (tipo == 1 && ((float*)coluna)[i] <= ((float*)coluna)[pivoIndex]) condicaoEsquerda = true;
if (tipo == 2 && strcmp((char**)coluna[i], ((char**)coluna)[pivoIndex]) <= 0) condicaoEsquerda = true;
if (tipo == 3 && ((long long*)coluna)[i] <= ((long long*)coluna)[pivoIndex]) condicaoEsquerda = true;

if (tipo == 0 && ((int*)coluna)[j] > ((int*)coluna)[pivoIndex]) condicaoDireita = true;
if (tipo == 1 && ((float*)coluna)[j] > ((float*)coluna)[pivoIndex]) condicaoDireita = true;
if (tipo == 2 && strcmp((char**)coluna[j], ((char**)coluna)[pivoIndex]) > 0) condicaoDireita = true;
if (tipo == 3 && ((long long*)coluna)[j] > ((long long*)coluna)[pivoIndex]) condicaoDireita = true;

if (condicaoEsquerda) i++;
```

Fonte: Própria (2025)

Se nenhuma das condições de avanço for satisfeita, significa que coluna[i] (que é maior que o pivô) e coluna[j] (que é menor ou igual ao pivô) estão fora de lugar e devem ser trocados. O bloco “else” executa a troca dos valores entre “coluna[i]”, utilizando novamente uma variável temporária e casting específico para cada tipo (este bloco “else” é o que executa a troca). É fundamental chamar a função “trocaLinhas” imediatamente para que a troca nos ponteiros das linhas de dados seja sincronizada. O contador de trocas é aumentado e os ponteiros “i” e “j” avançam ou recuam dar continuidade no processo.

Imagem 35 – Troca de valores entre as posições

```
else if (condicaoDireita) j--;
else {
    if (tipo == 0) { int t = ((int*)coluna)[i]; ((int*)coluna)[i] = ((int*)coluna)[j]; ((int*)coluna)[j] = t; }
    if (tipo == 1) { float t = ((float*)coluna)[i]; ((float*)coluna)[i] = ((float*)coluna)[j]; ((float*)coluna)[j] = t; }
    if (tipo == 2) { char* t = ((char**)coluna)[i]; ((char**)coluna)[i] = ((char**)coluna)[j]; ((char**)coluna)[j] = t; }
    if (tipo == 3) { long long t = ((long long*)coluna)[i]; ((long long*)coluna)[i] = ((long long*)coluna)[j]; ((long long*)coluna)[j] = t; }

    trocaLinhas(Linhas, i, j);
    trocas++;
    i++;
    j--;
}
```

Fonte: Própria (2025)

Após o laço, o pivô é posicionado corretamente. O bloco de código seguinte realiza a troca do pivô (em “pivoIndex”) com o elemento em “coluna[j]”, que é a posição

final do pivô. A “trocaLinha” é chamada novamente para mover a linha do pivô, e trocas é incrementado.

Imagem 36 – Troca final do pivô

```
if (tipo == 0) { int t = ((int*)coluna)[pivoIndex]; ((int*)coluna)[pivoIndex] = ((int*)coluna)[j]; ((int*)coluna)[j] = t; }
if (tipo == 1) { float t = ((float*)coluna)[pivoIndex]; ((float*)coluna)[pivoIndex] = ((float*)coluna)[j]; ((float*)coluna)[j] = t; }
if (tipo == 2) { char* t = ((char**)coluna)[pivoIndex]; ((char**)coluna)[pivoIndex] = ((char**)coluna)[j]; ((char**)coluna)[j] = t; }
if (tipo == 3) { long long t = ((long long*)coluna)[pivoIndex]; ((long long*)coluna)[pivoIndex] = ((long long*)coluna)[j]; ((long long*)coluna)[j] = t; }

trocaLinha(linhas, pivoIndex, j);
trocas++;
```

Fonte: Própria (2025)

Por fim, o processo se completa com as duas chamadas recursivas: uma para ordenar o segmento à esquerda do pivô (início a $j - 1$) e outra para o segmento à direita ($j + 1$ a fim), somando o total de trocas de volta ao resultado.

Imagem 37 – Chamada recursiva

```
trocas += QuickSortRec(linhas, coluna, tipo, inicio, j - 1);
trocas += QuickSortRec(linhas, coluna, tipo, j + 1, fim);

return trocas;
```

Fonte: Própria (2025)

A função “QuickSort” é apenas a interface que inicializa “QuickSortRec” com os limites de todo o vetor.

Imagem 38 – Função QuickSortRec

```
int QuickSort(char** linhas, void* coluna, int tipo, int n) {
    return QuickSortRec(linhas, coluna, tipo, 0, n - 1);
}
```

Fonte: Própria (2025)

5.8 Merge Sort

No algoritmo MergeSort, a função “merge” é responsável pela fusão(conquista) unido dois sub-arrays ordenados em um único array ordenado. A função começa determinando os tamanhos dos sub-arrays esquerdo (n_1) e direito (n_2).

Imagem 39 – Determinando os tamanhos dos sub-arrays

```
char** Llinhas = (char**)malloc(n1 * sizeof(char*));
char** Rlinhas = (char**)malloc(n2 * sizeof(char*));

void* Lcoluna;
void* Rcoluna;
```

Fonte: Própria (2025)

Depois, os quatro arrays auxiliares são alocados: "Llinhas" e "Rlinhas", para os valores da coluna. Para assegurar que a memória alocada tenha adequado(sizeof(int),sizeof(float),etc.), a atribuição de Lcoluna e Rcoluna depende do tipo(o que é feito por meio de uma estrutura if/esle if/else).

Imagem 40 - Alocamento dinâmico na memória para os *arrays* temporários da coluna

```
if (tipo == 0) {
    Lcoluna = malloc(n1 * sizeof(int));
    Rcoluna = malloc(n2 * sizeof(int));
} else if (tipo == 1) {
    Lcoluna = malloc(n1 * sizeof(float));
    Rcoluna = malloc(n2 * sizeof(float));
} else if (tipo == 2) {
    Lcoluna = malloc(n1 * sizeof(char*));
    Rcoluna = malloc(n2 * sizeof(char*));
} else {
    Lcoluna = malloc(n1 * sizeof(long long));
    Rcoluna = malloc(n2 * sizeof(long long));
}
```

Fonte: Própria (2025)

Os próximos dois blocos for copiam os dados dos segmentos originais (linhas e coluna, de esquerda até direita) para esses arrays temporários, garantindo que os valores da coluna e os ponteiros das linhas permaneçam emparelhados. Os índices i, j, e k são inicializados: i e j para rastrear as posições nos arrays temporários esquerdo e direito, respectivamente, e k para a posição no array principal (coluna e linhas).

Imagem 41 – Módulo que copiam os dados do *array* principal


```

for (i = 0; i < n1; i++) {
    Llinhas[i] = Linhas[esquerda + i];
    if (tipo == 0) ((int*)Lcoluna)[i] = ((int*)coluna)[esquerda + i];
    if (tipo == 1) ((float*)Lcoluna)[i] = ((float*)coluna)[esquerda + i];
    if (tipo == 2) ((char**)Lcoluna)[i] = ((char**)coluna)[esquerda + i];
    if (tipo == 3) ((long long*)Lcoluna)[i] = ((long long*)coluna)[esquerda + i];
}

for (j = 0; j < n2; j++) {
    Rlinhas[j] = Linhas[meio + 1 + j];
    if (tipo == 0) ((int*)Rcoluna)[j] = ((int*)coluna)[meio + 1 + j];
    if (tipo == 1) ((float*)Rcoluna)[j] = ((float*)coluna)[meio + 1 + j];
    if (tipo == 2) ((char**)Rcoluna)[j] = ((char**)coluna)[meio + 1 + j];
    if (tipo == 3) ((long long*)Rcoluna)[j] = ((long long*)coluna)[meio + 1 + j];
}

i = 0; j = 0; k = esquerda;

```

Fonte: Própria (2025)

O laço principal “while (i < n1 && j < n2)” realiza a fusão ordenada. A flag booleana menor é usada para determinar se o elemento em “Lcoluna[i]” é menor ou igual ao elemento em Rcoluna[j] (o que é determinado por uma série de “if” encadeados). Novamente, o casting e “strcmp” são essenciais aqui. Se o elemento esquerdo for o menor, ele é copiado de volta para a posição “k” do array principal (linhas[k] e coluna[k]), e “i” avança. Caso contrário(dentro de um bloco “else”), o elemento direito é copiado e j avança. Em ambos os casos, k avança, e o ponteiro (*trocas) é incrementado, contabilizando cada elemento copiado de volta ao array final.

Imagem 42 – Estrutura while

```

while (i < n1 && j < n2) {
    bool menor = false;

    if (tipo == 0 && ((int*)Lcoluna)[i] <= ((int*)Rcoluna)[j]) menor = true;
    if (tipo == 1 && ((float*)Lcoluna)[i] <= ((float*)Rcoluna)[j]) menor = true;
    if (tipo == 2 && strcmp(((char**)Lcoluna)[i], ((char**)Rcoluna)[j]) <= 0) menor = true;
    if (tipo == 3 && ((long long*)Lcoluna)[i] <= ((long long*)Rcoluna)[j]) menor = true;

    if (menor) {
        Linhas[k] = Llinhas[i];
        if (tipo == 0) ((int*)coluna)[k] = ((int*)Lcoluna)[i];
        if (tipo == 1) ((float*)coluna)[k] = ((float*)Lcoluna)[i];
        if (tipo == 2) ((char**)coluna)[k] = ((char**)Lcoluna)[i];
        if (tipo == 3) ((long long*)coluna)[k] = ((long long*)Lcoluna)[i];
        i++;
    } else {
        Linhas[k] = Rlinhas[j];
        if (tipo == 0) ((int*)coluna)[k] = ((int*)Rcoluna)[j];
        if (tipo == 1) ((float*)coluna)[k] = ((float*)Rcoluna)[j];
        if (tipo == 2) ((char**)coluna)[k] = ((char**)Rcoluna)[j];
        if (tipo == 3) ((long long*)coluna)[k] = ((long long*)Rcoluna)[j];
        j++;
    }
    (*trocas)++; // ? conta a troca
    k++;
}

```

Fonte: Própria (2025)

Finalmente, os dois laços “while” finais tratam os elementos restantes. Se acaso um dos sub-arrays temporários se esgote antes, os elementos restantes do outro sub-array (que já estão ordenados) são simplesmente copiados em sequência para o final do array principal, e o número de trocas é incrementado a cada cópia feita.

Imagem 43 – Copiando os elementos restantes

```

while (i < n1) {
    Linhas[k] = Llinhas[i];
    if (tipo == 0) ((int*)coluna)[k] = ((int*)Lcoluna)[i];
    if (tipo == 1) ((float*)coluna)[k] = ((float*)Lcoluna)[i];
    if (tipo == 2) ((char**)coluna)[k] = ((char**)Lcoluna)[i];
    if (tipo == 3) ((long long*)coluna)[k] = ((long long*)Lcoluna)[i];
    i++;
    k++;
    (*trocas)++; // ? conta a troca
}

while (j < n2) {
    Linhas[k] = Rlinhas[j];
    if (tipo == 0) ((int*)coluna)[k] = ((int*)Rcoluna)[j];
    if (tipo == 1) ((float*)coluna)[k] = ((float*)Rcoluna)[j];
    if (tipo == 2) ((char**)coluna)[k] = ((char**)Rcoluna)[j];
    if (tipo == 3) ((long long*)coluna)[k] = ((long long*)Rcoluna)[j];
    j++;
    k++;
    (*trocas)++; // ? conta a troca
}

```

Fonte: Própria (2025)

A função conclui com a etapa fundamental de liberação da memória, utilizando “free()” em todos os quatro arrays alocados (Llinha, Rlinhas, Lcoluna e Rcoluna), evitando vazamento de memória.

Imagem 44 – Liberação das memórias

```
free(Llinhas);
free(Rlinhas);
free(Lcoluna);
free(Rcoluna);
```

Fonte: Própria (2025)

As funções mergeSort e MergeSort gerenciam a recursão e a inicialização.

Imagem 45 - Recursão de divisão

```
void mergeSort(char** linhas, void* coluna, int tipo, int esquerda, int direita, int* trocas) {
    if (esquerda < direita) {
        int meio = esquerda + (direita - esquerda) / 2;
        mergeSort(linhas, coluna, tipo, esquerda, meio, trocas);
        mergeSort(linhas, coluna, tipo, meio + 1, direita, trocas);
        merge(linhas, coluna, tipo, esquerda, meio, direita, trocas);
    }
}

int MergeSort(char** linhas, void* coluna, int tipo, int n) {
    int trocas = 0;
    mergeSort(linhas, coluna, tipo, 0, n - 1, &trocas);
    return trocas;
}
```

Fonte: Própria (2025)

5.9 Função Main

A função “main()” é o núcleo do programa e seu principal objetivo é coordenar todas as etapas do processamento dos dados. É por meio dela onde ocorre toda interação do usuário com o programa: Opção de selecionar um arquivo CSV para leitura pelo programa, seleção de qual algoritmo de ordenação utilizar, identificação da coluna selecionada e aplicação do método de ordenação escolhido nas etapas anteriores. Assim, a função “main()” funciona como um ponto de integração dos diversos módulos e funções que compõem o código, organizando o fluxo de execução e garantindo o funcionamento adequado do programa como um todo.

O programa inicia apresentando um menu interativo, no qual, será disponibilizado duas opções arquivos em csv, denominados “focos_br_mg_ref_2023.csv” e “focos_br_mg_ref_2024.csv” que apresentam dados verídicos sobre focos de incêndio documentados no estado de Minas Gerais. A escolha do arquivo é realizada por meio de um número entre 1 e 2 inserido pelo usuário, após a seleção de qual arquivo utilizar, é chamado a função “fopen()”, que fará a leitura. Se o arquivo não for localizado, o programa exibe uma mensagem de erro por meio da função “perror()”, e encerra a sua execução de forma segura.

Imagem 46 - Menu de escolha do arquivo CSV

```
int main(void) {
    int opcaoArquivo;
    printf("Escolha o conjunto de dados:\n");
    printf("1 - Dados de 2023\n");
    printf("2 - Dados de 2024\n");
    printf("Opcao: ");
    scanf("%d", &opcaoArquivo);

    const char* arquivo2023 = "focos_br_mg_ref_2023.csv";
    const char* arquivo2024 = "focos_br_mg_ref_2024.csv";

    FILE* arquivo1 = NULL;
    FILE* arquivo2 = NULL;
```

Fonte: Própria (2025)

Imagem 47 – Switch/case para leitura do arquivo selecionado

```
switch (opcaoArquivo) {
    case 1:
        arquivo1 = fopen(arquivo2023, "r");
        if (!arquivo1) {
            perror("Erro ao abrir arquivo de 2023");
            return 1;
        }
        break;

    case 2:
        arquivo1 = fopen(arquivo2024, "r");
        if (!arquivo1) {
            perror("Erro ao abrir arquivo de 2024");
            return 1;
        }
        break;

    default:
        printf("Opcao invalida!\n");
        return 1;
}
```

Fonte: Própria (2025)

Depois que o arquivo é aberto, o programa cria inicialmente um vetor temporário denominado “buffer”, com tamanho adequado para guardar uma linha inteira do arquivo. Esse vetor atua como uma área de leitura temporária, onde cada linha do arquivo CSV é carregada antes de ser processada. A seguir, declara-se um

ponteiro duplo de caracteres (“char** linhas”) para armazenar todas as linhas lidas, e uma variável inteira denominado contador, que contabiliza o número de linhas processadas até aquele ponto. Depois de declarar estas variáveis, o programa executa o comando “while (fgets(buffer, sizeof(buffer), arquivo))” que lê o conteúdo do CSV linha por linha armazena cada segmento de texto no buffer até atingir o final do arquivo. Cada linha do arquivo CSV é armazenada no vetor dinâmico de strings “linhas”, mantendo todas as informações contidas nele. Todo o processo é executado de forma dinâmica com a função “realloc()”, que permite a realocação de memória conforme a quantidade de registros lidos. Essa abordagem assegura que o programa possa processar arquivos de variados tamanhos sem precisar de uma alocação de memória fixa, proporcionando mais flexibilidade e eficácia na leitura dos dados. Por último, a função “fclose(arquivo)” é chamada para encerrar o arquivo e liberar os recursos do sistema. Ao final desta fase, todas as linhas do arquivo CSV estarão carregadas na memória, disponíveis para serem processadas e organizadas de acordo com a preferência do usuário.

Imagem 48 – Laço que armazena dados do CSV no vetor linhas

```
char buffer[65536];
char** linhas = NULL;
int contador = 0;

while (fgets(buffer, sizeof(buffer), arquivo1)) {
    buffer[strcspn(buffer, "\r\n")] = '\0';
    linhas = (char**)realloc(linhas, (contador + 1) * sizeof(char*));
    linhas[contador] = (char*)malloc(strlen(buffer) + 1);
    strcpy(linhas[contador], buffer);
    contador++;
}
fclose(arquivo1);

if (arquivo2 != NULL) {
    while (fgets(buffer, sizeof(buffer), arquivo2)) {
        buffer[strcspn(buffer, "\r\n")] = '\0';
        linhas = (char**)realloc(linhas, (contador + 1) * sizeof(char*));
        linhas[contador] = (char*)malloc(strlen(buffer) + 1);
        strcpy(linhas[contador], buffer);
        contador++;
    }
    fclose(arquivo2);
}
```

Fonte: Própria (2025)

Posteriormente, o programa define quais colunas poderão ser ordenadas e seus tipos de dados de cada uma. Essa definição é executada através de dois vetores: um contendo os nomes das colunas e outro contendo a indicação de seus tipos. O

tipo de dados é definido por meio da numeração, sendo respectivamente, o valor 0 para valores de número inteiros, o valor 1 para números do tipo float, o valor 2 para a texto(Strings), o valor 3 para data, que é transformado em um valor numérico para simplificar a comparação durante a ordenação, e também o valor -1 para as colunas que não podem ser ordenadas. Com essa estrutura, o programa pode reconhecer automaticamente o formato da informação e aplicar o tratamento adequado em cada caso.

Imagem 49 – Vetores que define os nomes e os valores da coluna

```
const char* nomesColunas[] = {"id_bdq", "foco_id", "lat", "lon", "data_pas", "pais", "estado", "municipio", "bioma"};
int tiposColunas[] = {-1, -1, 1, 1, 3, -1, -1, 2, 2};
int numColunas = 9;
```

Fonte: Própria (2025)

Em seguida, o sistema apresenta ao usuário um novo menu, em que ele precisará escolher qual algoritmo de ordenação usar. São disponibilizados 8 métodos, consecutivamente: “*Selection Sort*”, “*Bubble Sort*”, “*Insertion Sort*”, “*Cocktail Shaker Sort*”, “*Quick Sort*”, “*Merge Sort*”, “*Shell Sort*” e “*Heap Sort*”. Depois de selecionar o algoritmo, o usuário deve indicar o nome da coluna que deseja ordenar, como “*latitude*”, “*data*” ou “*município*”. Após escolher a coluna, o programa determina o índice e a categoria dessa coluna para processar os dados de maneira adequada.

Imagem 50 – Menu para escolha do algoritmo de ordenação e escolha da coluna a ser ordenada

```
int escolha;
printf("Escolha o algoritmo de ordenacao:\n");
printf("1 - Selection Sort\n");
printf("2 - Bubble Sort\n");
printf("3 - Insertion Sort\n");
printf("4 - Cocktail Shaker Sort\n");
printf("5 - Quick Sort\n");
printf("6 - Merge Sort\n");
printf("7 - Shell Sort\n");
printf("8 - Heap Sort\n");
printf("Opcao: ");
scanf("%d", &escolha);

char nomeEscolhido[100];
printf("\nColunas ordenaveis: lat, lon, data_pas, municipio, bioma\n");
printf("Digite o nome da coluna: ");
scanf("%99s", nomeEscolhido);
```

Fonte: Própria (2025)

Imagem 51 – Laço que recebe os valores do scan menu para a escolha da coluna

```
int indiceColuna = -1;
int i;
for (i = 0; i < numColunas; i++) {
    if (strcmp(nomeEscolhido, nomesColunas[i]) == 0) {
        indiceColuna = i;
        break;
    }
}

if (indiceColuna == -1 || tiposColunas[indiceColuna] == -1) {
    printf("Coluna invalida ou nao ordenavel!\n");
    return 1;
}
```

Fonte: Própria (2025)

Antes de iniciar a execução do algoritmo selecionado, é declarado um vetor auxiliar chamado “coluna” que contém apenas o valor da coluna designada. Esse vetor é preenchido conforme o tipo de dado correspondente e funciona como base para a comparação dos elementos durante a ordenação. Para as colunas de data, o programa emprega a função “dataParaNumero()”, que converte o formato textual “YYYY-MM-DD HH:MM:SS” em um valor numérico do tipo long long. Isso permite a comparação direta entre várias datas. O preenchimento desse vetor é realizado dentro de um laço “for” que percorre todas as linhas previamente armazenadas na memória. Em cada iteração, a linha atual é temporariamente copiada usando a função “_strdup()”, e a função “strtok()” divide a linha em partes menores, separando os elementos com base na vírgula, que é o delimitador padrão para arquivos CSV. A função percorre os campos até localizar a coluna correspondente ao índice selecionado e, em seguida, converte o valor encontrado para o tipo de dado correto: inteiros com “atoi()”, números reais com “atof()”, textos com “_strdup()” e datas com a função “dataParaNumero()” que transforma o formato de texto da data para um número de fácil comparação. Em conclusão, a cópia temporária é libertada com “free(temp)”, para impedir o desperdício de memória. Essa fase é crucial para a preparação dos dados a serem ordenados, assegurando que o programa opere com informações corretamente formatadas e convertidas para o tipo apropriado.

Imagem 52 – Alocação e processamento de dados de coluna

```

int tipoColuna = tiposColunas[indiceColuna];

void* coluna = NULL;
if (tipoColuna == 0) coluna = malloc(contador * sizeof(int));
if (tipoColuna == 1) coluna = malloc(contador * sizeof(float));
if (tipoColuna == 2) coluna = malloc(contador * sizeof(char*));
if (tipoColuna == 3) coluna = malloc(contador * sizeof(long long));

for (i = 0; i < contador; i++) {
    char* temp = _strdup(linhas[i]);
    char* token = strtok(temp, ",");
    int j;
    for (j = 0; j < indiceColuna; j++)
        token = strtok(NULL, ",");
    if (tipoColuna == 0)
        ((int*)coluna)[i] = atoi(token);
    else if (tipoColuna == 1)
        ((float*)coluna)[i] = atof(token);
    else if (tipoColuna == 2)
        ((char**)coluna)[i] = _strdup(token);
    else if (tipoColuna == 3)
        ((long long*)coluna)[i] = dataParaNumero(token);
    free(temp);
}

```

Fonte: Própria (2025)

Com todas as informações organizadas, o software executa o algoritmo escolhido. A seleção do método é administrada por um bloco de instruções switch-case, que invoca a função correspondente ao número inserido pelo usuário. Cada função recebe como parâmetros o vetor de linhas completas (“linhas”), o vetor com os valores da coluna a ser ordenada (“coluna”), o tipo de dado (“tipoColuna”) e a quantidade total de registros (“contador”). O retorno de cada função é a quantidade de trocas feitas durante a ordenação, um valor usado como métrica para comparar o desempenho de cada algoritmo.

Imagem 53 - Switch/case para chamar o algoritmo selecionado

```

int trocas = 0;
switch (escolha) {
    case 1: trocas = SelectionSort(linhas, coluna, tipoColuna, contador); break;
    case 2: trocas = BubbleSort(linhas, coluna, tipoColuna, contador); break;
    case 3: trocas = InsertionSort(linhas, coluna, tipoColuna, contador); break;
    case 4: trocas = CocktailShakerSort(linhas, coluna, tipoColuna, contador); break;
    case 5: trocas = QuickSort(linhas, coluna, tipoColuna, contador); break;
    case 6: trocas = MergeSort(linhas, coluna, tipoColuna, contador); break;
    case 7: trocas = ShellSort(linhas, coluna, tipoColuna, contador); break;
    case 8: trocas = HeapSort(linhas, coluna, tipoColuna, contador); break;
    default: printf("Opcao invalida!\n"); return 1;
}

```

Fonte: Própria (2025)

Depois de concluída a ordenação, o programa executa mais um laço “for” para percorrer todas as linhas armazenadas, no qual, cada elemento do vetor “linha” é apresentado na tela por meio do comando de saída “printf()”, essa impressão exibe o conteúdo completo do arquivo já organizado de acordo com o critério estabelecido pelo usuário, possibilitando uma visualização direta do resultado alcançado. Posteriormente, dentro do mesmo laço, a função “free(linhas[i])” é utilizada para liberar cada posição do vetor, garantindo que a memória alocada dinamicamente para cada linha seja devidamente deslocada após o uso. Além de exibir o conteúdo do arquivo todo ordenado, o programa também apresenta o número de total de linhas tratadas e ainda conta o total de trocas feitas durante o processo de ordenação, informações estas que auxiliam a confirmar a precisão do algoritmo e a analisar sua eficácia em diversas condições.

Imagem 54 - Laço para impressão dos dados ordenado, printf que mostra o número de linhas total e o total de trocas, Laço para liberar memória do vetor coluna

```
printf("\n--- Linhas ordenadas ---\n");
for (i = 0; i < contador; i++) {
    printf("%s\n", linhas[i]);
    free(linhas[i]);
}

printf("\nTotal de linhas ordenadas: %d\n", contador);
printf("Total de trocas realizadas: %d\n", trocas);

if (tipoColuna == 2) {
    char** arr = (char**)coluna;
    for (i = 0; i < contador; i++)
        free(arr[i]);
}
```

Fonte: Própria (2025)

Por fim, o programa libera toda a memória alocada dinamicamente durante a execução por meio da função “free()”. Mas, quando a coluna processada é do tipo texto, cada string armazenada é deslocada individualmente antes da liberação completa do vetor “coluna”. Na sequência, para evitar vazamentos de memória (memory leaks), tanto o vetor auxiliar quanto o vetor principal “linhas” são liberados. Esse procedimento garante que o sistema conclua sua execução de maneira segura e eficaz, preservando a estabilidade e o desempenho otimizado do programa em C.

Imagem 55 - Função free para liberar memória dinâmica

```
free(coluna);  
free(linhas);
```

Fonte: Própria (2025)

Em síntese, a função “main()” desempenha um papel fundamental para a arquitetura do programa, sendo o elemento responsável por coordenar as etapas de entrada de dados. Além de proporcionar uma interface simples e interativa para o usuário, sua estrutura modular e bem-organizada possibilita a integração eficaz dos diferentes algoritmos de ordenação implementados. Dessa forma, a função “main()” não desempenha não apenas um papel operacional, mas também um papel didático, possibilitando a observação e comparação do comportamento e da eficiência dos algoritmos de maneira objetiva.

6 CONCLUSÃO

6.1 Discussão dos Resultados Obtidos

O objetivo geral deste trabalho foi desenvolver um programa capaz de ordenar as bases de dados geográficos com diferentes algoritmos de ordenação, a fim de permitir uma análise mais rápida e precisa, sendo essa a primeira etapa do processamento de dados, preparando-os para as técnicas de machine learning ou deep learning.

O programa foi estruturado com base em um menu, no qual é possível escolher entre a base de dados de 2023 ou a de 2024, assim garantindo uma análise mais detalhada, já que há a separação das bases por seu respectivo ano de levantamento, e foi implementado um sistema de tratamento de erro, caso o arquivo possua algum erro ou o usuário escolha uma opção inválida. Ao selecionar a base de dados desejada, é requerido o método de ordenação na qual deverá ser aplicado sobre os dados, e as opções são apresentadas ao usuário enumeradas de 1 a 8 e com o nome do respectivo método de ordenação, as opções são: Selection Sort, Bubble Sort, Insertion Sort, Cocktail Shaker Sort, Quick Sort, Merge Sort, Shell Sort e Heap Sort. Após a escolha do método de ordenação, o programa apresenta ao usuário as colunas disponíveis para a ordenação, sendo elas: lat, lon, data_pas, município e bioma, assim que for definida uma coluna, os dados da base serão arranjados na mesma com base no método de ordenação escolhido, e além de mostrar os dados ordenados na tela, é possível ter acesso a quantidade de linhas ordenadas e trocas realizadas pelo algoritmo de ordenação optado.

A partir do acesso ao número de trocas e ao tempo de execução, percebemos, após diversos testes, que alguns algoritmos tendem ser mais eficiente que os outros, o “Bubble Sort”, “Insertion Sort” e o “Cocktail Shaker Sort” que apesar de fazer a ordenação completa dos arquivos, eles foram os algoritmos se demonstraram menos eficientes em todos os quesitos, efetuando milhões de trocas durante a ordenação em qualquer tipo de dado e o tempo de execução em comparação com os outros algoritmos, são os mais longos, por conta sua extensa demora para fazer as trocas entre as linhas. Mesmo com alguns algoritmos não demonstrando muito eficiente para o seu objetivo, o outro se comprovaram bem eficiente para ordenação dos arquivo CSV, o “Quick Sort”, “Merge Sort”, “Shell Sort” e o “Heap Sort” foram os que

se provaram mais eficientes nos quesito de tempo de execução, levando menos 2 segundos na ordenação de todas as linhas para qualquer tipo de dado, efetuando também bem menos trocas entre as linhas, sendo no número entre 20 mil a 100 mil trocas para todos os tipos, tendo apenas o “Shell Sort” e o “Quick Sort” que conseguiram fazer menos de 15 mil trocas usando a coluna “bioma” para fazer a ordenação. Embora esses algoritmos tenham se saído bem na avaliação, o “Selection Sort” foi o melhor e mais eficiente entre todos os outros algoritmos, pois apesar de ter o tempo de execução maior um pouco que o “Quick Sort”, “Merge Sort”, “Shell Sort” e o “Heap Sort”, ele acaba sendo muito mais rápido que o “Bubble Sort”, “Insertion Sort” e o “Cocktail Shaker Sort”, e durante os testes, constatamos que o “Selection Sort” foi o único algoritmo que efetuou o número de trocas menor que o número total de linhas presente no arquivo CSV. Isso mostra que o “Selection Sort” mantém um bom desempenho mesmo em grandes bases de dados, demonstrando uma relação direta entre simplicidade e eficiência prática. Isso evidencia seu potencial em situações em que a estabilidade e a previsibilidade de desempenho são fundamentais.

6.2 Impacto do Algoritmo de Ordenação para Visualização de Fenômenos do Meio Ambiente

No contexto da aplicação dos algoritmos aos dados sobre as queimadas do estado de Minas Gerais, se mostrou evidente a importância das estruturas de dados na organização, processamento e interpretação de informações ambientais. O ato de ordenar e estruturar adequadamente grandes volumes de registros possibilitou uma visão sistemática e confiável dos focos de incêndio, contribuindo para o entendimento de como, quando e onde esses eventos ocorrem.

A ordenação segundo os critérios data, bioma, município e precipitação permitiu uma visão mais clara e estruturada dos dados, facilitando a observação de padrões temporais e espaciais nas ocorrências de queimadas, revelando regiões com maior concentração de focos e também a identificação de períodos do ano com maior propensão a incêndios, o que pode servir de base para planejamento preventivo e políticas públicas ambientais, contribuindo para a redução de riscos e a mitigação de possíveis desastres ecológicos. Esse tipo de processamento inicial é essencial para análises mais avançadas, nas quais técnicas de machine learning e deep learning podem ser aplicadas. Desta forma, o projeto atua como um pré-processador de dados

ambientais, servindo de base para futuras soluções automatizadas de detecção e previsão de queimadas, o que representa um avanço relevante no campo da computação aplicada à sustentabilidade.

Além disso, o uso de dados reais de fontes oficiais, como os disponibilizados pelo Instituto Nacional de Pesquisas Espaciais (INPE), reforça a relevância prática e social do trabalho, demonstrando uma forte conexão entre o ambiente acadêmico e os desafios reais enfrentados pelo país.

O desenvolvimento deste sistema de análise de performance, portanto, vai além de um exercício técnico. A capacidade de utilizar recursos computacionais para compreender fenômenos naturais complexos, demonstra o potencial transformador da tecnologia quando aplicada de forma ética e responsável.

6.3 Relação de Machine e Deep Learning com Algoritmos de Ordenação

Como visto anteriormente, o Machine Learning é um campo da inteligência artificial que permite que computadores aprendam padrões e tomem decisões com base em dados, sem serem explicitamente programados para isso. O aprendizado de máquina utiliza algoritmos e modelos estatísticos para analisar grandes volumes de dados, identificar padrões, prever comportamentos e aprimorar processos. Essa metodologia é frequentemente utilizada em campos como reconhecimento de voz, recomendação de conteúdo, diagnóstico médico e previsão de tendências. O conceito fundamental do Machine Learning é possibilitar que os sistemas aprimorem constantemente seu desempenho à medida que são expostos a novos dados, tornando-se mais precisos e autônomos com o passar do tempo.

Quanto ao Deep Learning, é uma subárea do aprendizado de máquina que usa redes neurais artificiais de múltiplas camadas para processar dados e extrair características complexas. Baseado no funcionamento do cérebro humano, o Deep Learning consegue reconhecer padrões em níveis cada vez mais abstratos, possibilitando progressos consideráveis em campos como visão computacional, processamento de linguagem natural e reconhecimento de imagens. O principal diferencial dessa técnica é sua habilidade de aprender diretamente de dados não estruturados, como imagens, sons e textos, eliminando, em muitos casos, a necessidade de intervenção humana na fase de extração de características. Dessa

forma, o Deep Learning se estabeleceu como uma das tecnologias mais promissoras dos dias atuais.

A relação entre algoritmos de ordenação, machine learning e deep learning evidencia que os princípios clássicos da computação continuam sendo fundamentais, mesmo nas tecnologias mais sofisticadas da atualidade. Os algoritmos de ordenação, que foram originalmente criados para organizar dados de forma eficiente, continuam sendo fundamentais nos processos de aprendizado automatizado, assegurando que as informações sejam estruturadas e facilmente acessíveis para os modelos de inteligência artificial.

No campo do machine learning, a ordenação é utilizada em diversas etapas, desde o pré-processamento até a análise dos resultados. Ela possibilita a organização e manipulação dos dados de maneira a melhorar o desempenho dos algoritmos de aprendizado. O tempo de treinamento, a precisão das previsões e a habilidade do modelo em generalizar padrões são influenciados diretamente pela forma como os dados são organizados. Dessa forma, a ordenação não é mais apenas um procedimento técnico, mas se torna um elemento essencial do processo de aprendizado.

Já no contexto do deep learning, a ordenação assume um papel ainda mais fundamental. As redes neurais profundas processam grandes quantidades de dados e exigem que essas informações estejam corretamente estruturadas para que o aprendizado seja eficaz. Ademais, a ordenação é empregada em processos de classificação, priorização e ranqueamento de resultados, possibilitando que as redes aprendam a reconhecer quais dados são mais significativos em um contexto específico.

Dessa forma, nota-se que os algoritmos de ordenação atuam como um elo entre a organização de dados e o aprendizado inteligente. Embora o machine learning e o deep learning ofereçam a habilidade de adaptação e aprendizado autônomo, os algoritmos de ordenação proporcionam a fundamentação lógica e estrutural que embasa esse processo. A integração entre essas áreas reforça a importância de compreender os fundamentos da computação para o desenvolvimento de sistemas de inteligência artificial mais eficientes, precisos e escaláveis.

Com o avanço da tecnologia e o crescimento exponencial do volume de dados disponíveis, a demanda por combinar uma ordenação eficiente com métodos de aprendizado profundo se torna ainda mais clara. A maneira como os dados são

estruturados afeta diretamente a rapidez do processamento e a habilidade de um modelo em identificar padrões complexos. Portanto, a investigação e o desenvolvimento de métodos híbridos que combinam estratégias de ordenação e aprendizado são fundamentais para lidar com os desafios da inteligência artificial.

Em síntese, a relação entre algoritmos de ordenação, machine learning e deep learning representa uma conexão entre o passado e o futuro da computação. Enquanto a ordenação carrega os princípios fundamentais da lógica e da eficiência algorítmica, o aprendizado de máquina e o aprendizado profundo representam a evolução rumo à autonomia e à inteligência computacional. Essa interdependência mostra que o avanço tecnológico depende não apenas da criação de novas técnicas, mas também da valorização e do aprimoramento contínuo dos fundamentos que sustentam toda a ciência da computação.

6.4 Importância dos Algoritmos de Ordenação para a Ciência da Computação

Segundo Souza, Ricarte e Lima (2017) a grande quantidade de dados se torna um problema a ser solucionado. Desse jeito, a implementação de algoritmos de ordenação para melhor organizar essas informações se faz necessária. Do ponto de vista acadêmico, o estudo de algoritmos de ordenação permite a análise formal de complexidade computacional (notação Big O), otimização de estruturas de dados e o desenvolvimento de novos paradigmas algorítmicos.

A aprendizagem sobre algoritmos de ordenação para o cientista da computação vai além de entender sobre ordenação de dados, é uma abertura para o pensamento computacional e uma ferramenta essencial que sustenta praticamente todos os sistemas computacionais modernos, desde bancos de dados até aprendizado de máquina e processamento de big data.

6.5 Considerações Finais

O estudo dos algoritmos de ordenação e das técnicas de aprendizado de máquina e aprendizado profundo evidencia a evolução contínua da ciência da computação, desde seus fundamentos lógicos até as aplicações mais complexas da

inteligência artificial. Algoritmos de ordenação como Bubble Sort, Insertion Sort, Selection Sort, Quick Sort, Merge Sort, Shell Sort, Cocktail Shaker Sort e Heap Sort constituem a base fundamental para a maioria das operações computacionais. Cada um desses métodos, com seus pontos fortes e fracos, ajuda a entender as várias estratégias de eficiência, uso de memória e complexidade de execução.

O Bubble Sort, o Insertion Sort e o Selection Sort são exemplos clássicos de algoritmos simples, ideais para fins educacionais e para compreender a lógica de comparação e troca entre elementos. Já algoritmos mais sofisticados, como Quick Sort, Merge Sort e Heap Sort, mostram como o uso de paradigmas como divisão e conquista pode diminuir significativamente o tempo de processamento, tornando a ordenação de grandes volumes de dados mais eficiente. Por outro lado, o Shell Sort e o Cocktail Shaker Sort exemplificam variações e melhorias das técnicas convencionais, demonstrando como mudanças estruturais sutis podem resultar em melhorias significativas de desempenho em certas condições.

Esses algoritmos, embora desenvolvidos há décadas, continuam sendo indispensáveis para o funcionamento de sistemas modernos. Além de otimizar a manipulação de dados, eles oferecem a estrutura lógica necessária para operações mais complexas, como as usadas em Machine Learning e Deep Learning. A eficiência na organização e no acesso aos dados é um pré-requisito fundamental para o treinamento de modelos inteligentes, evidenciando a interdependência entre os princípios da computação e as abordagens contemporâneas de inteligência artificial.

No campo do Machine Learning, a capacidade de identificar padrões e aprender a partir de dados estruturados ou não estruturados depende fortemente da qualidade e da organização das informações utilizadas. Por outro lado, o Deep Learning, com suas redes neurais profundas, eleva esse conceito a um nível superior, pois consegue processar e interpretar dados complexos de forma autônoma, alinhando o comportamento computacional às formas humanas de percepção e raciocínio.

A aplicação dessas técnicas ao processamento e reconhecimento de imagens representa um dos avanços mais notáveis da computação contemporânea. O aprendizado profundo possibilita que sistemas reconheçam objetos, rostos, padrões e até emoções em imagens e vídeos, com uma precisão antes considerada impossível, por meio de arquiteturas de redes neurais convolucionais e outras estruturas especializadas. Essa combinação dos princípios clássicos de ordenação

com as técnicas modernas de aprendizado automatizado permite o tratamento de grandes volumes de informações visuais, organizando, classificando e interpretando imagens de maneira eficaz.

Conclui-se, portanto, que a união entre os fundamentos algorítmicos clássicos e as tecnologias de aprendizado de máquina e aprendizado profundo forma a base do desenvolvimento da computação moderna. Enquanto os algoritmos de ordenação oferecem a estrutura e a eficiência necessárias para manipulação de dados, o Machine Learning e o Deep Learning ampliam as fronteiras da inteligência computacional, permitindo que máquinas não apenas processem, mas também compreendam e aprendam com as informações disponíveis. Essa convergência representa o equilíbrio entre a teoria e a prática, entre o raciocínio lógico e a capacidade adaptativa, e consolida o papel da computação como uma das ciências mais dinâmicas e transformadoras do mundo atual.

7 RELATÓRIO COM AS LINHAS DE CÓDIGO DO PROGRAMA

```

#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
#include <ctype.h>

long long dataParaNumero(const char* data) {
    char buffer[15];
    int k = 0;
    int i;
    for (i = 0; data[i] != '\0' && k < 14; i++) {
        if (isdigit((unsigned char)data[i])) buffer[k++] = data[i];
    }
    buffer[k] = '\0';
    return atoll(buffer);
}

void trocaLinha(char** linhas, int i, int j) {
    char* temp = linhas[i];
    linhas[i] = linhas[j];
    linhas[j] = temp;
}

// --- Selection Sort ---
int SelectionSort(char** linhas, void* coluna, int tipo, int n) {
    int trocas = 0;
    int i, j;
    for (i = 0; i < n; i++) {
        int min = i;
        for (j = i + 1; j < n; j++) {

```

```

        bool menor = false;
        if (tipo == 0 && ((int*)coluna)[j] < ((int*)coluna)[min]) menor = true;
        if (tipo == 1 && ((float*)coluna)[j] < ((float*)coluna)[min]) menor = true;
        if (tipo == 2 && strcmp(((char**)coluna)[j], ((char**)coluna)[min]) < 0)
menor = true;
        if (tipo == 3 && ((long long*)coluna)[j] < ((long long*)coluna)[min]) menor
= true;

        if (menor) min = j;
    }
    if (min != i) {
        if (tipo == 0) { int t = ((int*)coluna)[i]; ((int*)coluna)[i] = ((int*)coluna)[min];
((int*)coluna)[min] = t; }
        if (tipo == 1) { float t = ((float*)coluna)[i]; ((float*)coluna)[i] =
((float*)coluna)[min]; ((float*)coluna)[min] = t; }
        if (tipo == 2) { char* t = ((char**)coluna)[i]; ((char**)coluna)[i] =
((char**)coluna)[min]; ((char**)coluna)[min] = t; }
        if (tipo == 3) { long long t = ((long long*)coluna)[i]; ((long long*)coluna)[i]
= ((long long*)coluna)[min]; ((long long*)coluna)[min] = t; }
        trocaLinha(linhas, i, min);
        trocas++;
    }
}

return trocas;
}

// --- Bubble Sort ---
int BubbleSort(char** linhas, void* coluna, int tipo, int n) {
    int trocas = 0;
    bool troca = true;
    while (troca) {
        int i;
        troca = false;
        for (i = 0; i < n - 1; i++) {
            bool precisa = false;
            if (tipo == 0 && ((int*)coluna)[i] > ((int*)coluna)[i + 1]) precisa = true;

```

```

        if (tipo == 1 && ((float*)coluna)[i] > ((float*)coluna)[i + 1]) precisa = true;
        if (tipo == 2 && strcmp(((char**)coluna)[i], ((char**)coluna)[i + 1]) > 0)
precisa = true;
        if (tipo == 3 && ((long long*)coluna)[i] > ((long long*)coluna)[i + 1])
precisa = true;
        if (precisa) {
            if (tipo == 0) { int t = ((int*)coluna)[i]; ((int*)coluna)[i] = ((int*)coluna)[i
+ 1]; ((int*)coluna)[i + 1] = t; }
            if (tipo == 1) { float t = ((float*)coluna)[i]; ((float*)coluna)[i] =
((float*)coluna)[i + 1]; ((float*)coluna)[i + 1] = t; }
            if (tipo == 2) { char* t = ((char**)coluna)[i]; ((char**)coluna)[i] =
((char**)coluna)[i + 1]; ((char**)coluna)[i + 1] = t; }
            if (tipo == 3) { long long t = ((long long*)coluna)[i]; ((long
long*)coluna)[i] = ((long long*)coluna)[i + 1]; ((long long*)coluna)[i + 1] = t; }
            trocaLinha(linhas, i, i + 1);
            troca = true;
            trocas++;
        }
    }
}
return trocas;
}

```

// --- Insertion Sort ---

```

int InsertionSort(char** linhas, void* coluna, int tipo, int n) {
    int trocas = 0;
    int i;
    for (i = 1; i < n; i++) {
        int j = i - 1;
        char* linhaChave = linhas[i];
        if (tipo == 0) {
            int chave = ((int*)coluna)[i];
            while (j >= 0 && ((int*)coluna)[j] > chave) {
                ((int*)coluna)[j + 1] = ((int*)coluna)[j];

```

```

        linhas[j + 1] = linhas[j];
        j--;
        trocas++;
    }
    ((int*)coluna)[j + 1] = chave;
} else if (tipo == 1) {
    float chave = ((float*)coluna)[i];
    while (j >= 0 && ((float*)coluna)[j] > chave) {
        ((float*)coluna)[j + 1] = ((float*)coluna)[j];
        linhas[j + 1] = linhas[j];
        j--;
        trocas++;
    }
    ((float*)coluna)[j + 1] = chave;
} else if (tipo == 2) {
    char* chave = ((char**)coluna)[i];
    while (j >= 0 && strcmp(((char**)coluna)[j], chave) > 0) {
        ((char**)coluna)[j + 1] = ((char**)coluna)[j];
        linhas[j + 1] = linhas[j];
        j--;
        trocas++;
    }
    ((char**)coluna)[j + 1] = chave;
} else if (tipo == 3) {
    long long chave = ((long long*)coluna)[i];
    while (j >= 0 && ((long long*)coluna)[j] > chave) {
        ((long long*)coluna)[j + 1] = ((long long*)coluna)[j];
        linhas[j + 1] = linhas[j];
        j--;
        trocas++;
    }
    ((long long*)coluna)[j + 1] = chave;
}
linhas[j + 1] = linhaChave;

```

```

    }
    return trocas;
}

// --- Cocktail Shaker Sort ---
int CocktailShakerSort(char** linhas, void* coluna, int tipo, int n) {
    int trocas = 0;
    bool troca = true;
    int inicio = 0, fim = n - 1;
    while (troca) {
        int i;
        troca = false;
        for (i = inicio; i < fim; i++) {
            bool precisa = false;
            if (tipo == 0 && ((int*)coluna)[i] > ((int*)coluna)[i + 1]) precisa = true;
            if (tipo == 1 && ((float*)coluna)[i] > ((float*)coluna)[i + 1]) precisa = true;
            if (tipo == 2 && strcmp(((char**)coluna)[i], ((char**)coluna)[i + 1]) > 0)
                precisa = true;
            if (tipo == 3 && ((long long*)coluna)[i] > ((long long*)coluna)[i + 1])
                precisa = true;
            if (precisa) {
                if (tipo == 0) { int t = ((int*)coluna)[i]; ((int*)coluna)[i] = ((int*)coluna)[i
+ 1]; ((int*)coluna)[i + 1] = t; }
                if (tipo == 1) { float t = ((float*)coluna)[i]; ((float*)coluna)[i] =
((float*)coluna)[i + 1]; ((float*)coluna)[i + 1] = t; }
                if (tipo == 2) { char* t = ((char**)coluna)[i]; ((char**)coluna)[i] =
((char**)coluna)[i + 1]; ((char**)coluna)[i + 1] = t; }
                if (tipo == 3) { long long t = ((long long*)coluna)[i]; ((long
long*)coluna)[i] = ((long long*)coluna)[i + 1]; ((long long*)coluna)[i + 1] = t; }
                trocaLinha(linhas, i, i + 1);
                troca = true;
                trocas++;
            }
        }
    }
    if (!troca) break;
}

```

```

    troca = false;
    fim--;
    for (i = fim; i > inicio; i--) {
        bool precisa = false;
        if (tipo == 0 && ((int*)coluna)[i - 1] > ((int*)coluna)[i]) precisa = true;
        if (tipo == 1 && ((float*)coluna)[i - 1] > ((float*)coluna)[i]) precisa = true;
        if (tipo == 2 && strcmp(((char**)coluna)[i - 1], ((char**)coluna)[i]) > 0)
            precisa = true;
        if (tipo == 3 && ((long long*)coluna)[i - 1] > ((long long*)coluna)[i]) precisa
            = true;

        if (precisa) {
            if (tipo == 0) { int t = ((int*)coluna)[i - 1]; ((int*)coluna)[i - 1] =
                ((int*)coluna)[i]; ((int*)coluna)[i] = t; }
            if (tipo == 1) { float t = ((float*)coluna)[i - 1]; ((float*)coluna)[i - 1] =
                ((float*)coluna)[i]; ((float*)coluna)[i] = t; }
            if (tipo == 2) { char* t = ((char**)coluna)[i - 1]; ((char**)coluna)[i - 1] =
                ((char**)coluna)[i]; ((char**)coluna)[i] = t; }
            if (tipo == 3) { long long t = ((long long*)coluna)[i - 1]; ((long
                long*)coluna)[i - 1] = ((long long*)coluna)[i]; ((long long*)coluna)[i] = t; }
            trocaLinha(linhas, i - 1, i);
            troca = true;
            trocas++;
        }
    }
    inicio++;
}
return trocas;
}

```

// --- Quick Sort ---

```

int QuickSortRec(char** linhas, void* coluna, int tipo, int inicio, int fim) {
    int trocas = 0;

    if (inicio >= fim) return trocas;

```

```

int pivolIndex = inicio;
int i = inicio + 1;
int j = fim;

while (i <= j) {
    bool condicaoEsquerda = false;
    bool condicaoDireita = false;

    if (tipo == 0 && ((int*)coluna)[i] <= ((int*)coluna)[pivolIndex])
condicaoEsquerda = true;
    if (tipo == 1 && ((float*)coluna)[i] <= ((float*)coluna)[pivolIndex])
condicaoEsquerda = true;
    if (tipo == 2 && strcmp(((char**)coluna)[i], ((char**)coluna)[pivolIndex]) <=
0) condicaoEsquerda = true;
    if (tipo == 3 && ((long long*)coluna)[i] <= ((long long*)coluna)[pivolIndex])
condicaoEsquerda = true;

    if (tipo == 0 && ((int*)coluna)[j] > ((int*)coluna)[pivolIndex]) condicaoDireita
= true;
    if (tipo == 1 && ((float*)coluna)[j] > ((float*)coluna)[pivolIndex])
condicaoDireita = true;
    if (tipo == 2 && strcmp(((char**)coluna)[j], ((char**)coluna)[pivolIndex]) > 0)
condicaoDireita = true;
    if (tipo == 3 && ((long long*)coluna)[j] > ((long long*)coluna)[pivolIndex])
condicaoDireita = true;

    if (condicaoEsquerda) i++;
    else if (condicaoDireita) j--;
    else {
        if (tipo == 0) { int t = ((int*)coluna)[i]; ((int*)coluna)[i] = ((int*)coluna)[j];
((int*)coluna)[j] = t; }
        if (tipo == 1) { float t = ((float*)coluna)[i]; ((float*)coluna)[i] =
((float*)coluna)[j]; ((float*)coluna)[j] = t; }

```



```

        if (tipo == 2) { char* t = ((char**)coluna)[i]; ((char**)coluna)[i] =
        ((char**)coluna)[j]; ((char**)coluna)[j] = t; }

```

```

        if (tipo == 3) { long long t = ((long long*)coluna)[i]; ((long long*)coluna)[i]
        = ((long long*)coluna)[j]; ((long long*)coluna)[j] = t; }

```

```

        trocaLinha(linhas, i, j);
        trocas++;
        i++;
        j--;
    }
}

```

```

    if (tipo == 0) { int t = ((int*)coluna)[pivoIndex]; ((int*)coluna)[pivoIndex] =
    ((int*)coluna)[j]; ((int*)coluna)[j] = t; }

```

```

    if (tipo == 1) { float t = ((float*)coluna)[pivoIndex]; ((float*)coluna)[pivoIndex] =
    ((float*)coluna)[j]; ((float*)coluna)[j] = t; }

```

```

    if (tipo == 2) { char* t = ((char**)coluna)[pivoIndex];
    ((char**)coluna)[pivoIndex] = ((char**)coluna)[j]; ((char**)coluna)[j] = t; }

```

```

    if (tipo == 3) { long long t = ((long long*)coluna)[pivoIndex]; ((long
    long*)coluna)[pivoIndex] = ((long long*)coluna)[j]; ((long long*)coluna)[j] = t; }

```

```

    trocaLinha(linhas, pivoIndex, j);
    trocas++;

```

```

    trocas += QuickSortRec(linhas, coluna, tipo, inicio, j - 1);
    trocas += QuickSortRec(linhas, coluna, tipo, j + 1, fim);

```

```

    return trocas;
}

```

```

int QuickSort(char** linhas, void* coluna, int tipo, int n) {
    return QuickSortRec(linhas, coluna, tipo, 0, n - 1);
}

```

```

}
// --- Merge Sort ---
void merge(char** linhas, void* coluna, int tipo, int esquerda, int meio, int direita,
int* trocas) {
    int i, j, k;
    int n1 = meio - esquerda + 1;
    int n2 = direita - meio;

    char** Llinhas = (char**)malloc(n1 * sizeof(char*));
    char** Rlinhas = (char**)malloc(n2 * sizeof(char*));

    void* Lcoluna;
    void* Rcoluna;

    if (tipo == 0) {
        Lcoluna = malloc(n1 * sizeof(int));
        Rcoluna = malloc(n2 * sizeof(int));
    } else if (tipo == 1) {
        Lcoluna = malloc(n1 * sizeof(float));
        Rcoluna = malloc(n2 * sizeof(float));
    } else if (tipo == 2) {
        Lcoluna = malloc(n1 * sizeof(char*));
        Rcoluna = malloc(n2 * sizeof(char*));
    } else {
        Lcoluna = malloc(n1 * sizeof(long long));
        Rcoluna = malloc(n2 * sizeof(long long));
    }

    for (i = 0; i < n1; i++) {
        Llinhas[i] = linhas[esquerda + i];
        if (tipo == 0) ((int*)Lcoluna)[i] = ((int*)coluna)[esquerda + i];
        if (tipo == 1) ((float*)Lcoluna)[i] = ((float*)coluna)[esquerda + i];
        if (tipo == 2) ((char**)Lcoluna)[i] = ((char**)coluna)[esquerda + i];
        if (tipo == 3) ((long long*)Lcoluna)[i] = ((long long*)coluna)[esquerda + i];
    }
}

```

```
}
```

```
for (j = 0; j < n2; j++) {
    Rlinhas[j] = linhas[meio + 1 + j];
    if (tipo == 0) ((int*)Rcoluna)[j] = ((int*)coluna)[meio + 1 + j];
    if (tipo == 1) ((float*)Rcoluna)[j] = ((float*)coluna)[meio + 1 + j];
    if (tipo == 2) ((char**)Rcoluna)[j] = ((char**)coluna)[meio + 1 + j];
    if (tipo == 3) ((long long*)Rcoluna)[j] = ((long long*)coluna)[meio + 1 + j];
}
```

```
i = 0; j = 0; k = esquerda;
```

```
while (i < n1 && j < n2) {
```

```
    bool menor = false;
```

```
    if (tipo == 0 && ((int*)Lcoluna)[i] <= ((int*)Rcoluna)[j]) menor = true;
```

```
    if (tipo == 1 && ((float*)Lcoluna)[i] <= ((float*)Rcoluna)[j]) menor = true;
```

```
    if (tipo == 2 && strcmp(((char**)Lcoluna)[i], ((char**)Rcoluna)[j]) <= 0)
```

```
menor = true;
```

```
    if (tipo == 3 && ((long long*)Lcoluna)[i] <= ((long long*)Rcoluna)[j]) menor
```

```
= true;
```

```
    if (menor) {
```

```
        linhas[k] = Llinhas[i];
```

```
        if (tipo == 0) ((int*)coluna)[k] = ((int*)Lcoluna)[i];
```

```
        if (tipo == 1) ((float*)coluna)[k] = ((float*)Lcoluna)[i];
```

```
        if (tipo == 2) ((char**)coluna)[k] = ((char**)Lcoluna)[i];
```

```
        if (tipo == 3) ((long long*)coluna)[k] = ((long long*)Lcoluna)[i];
```

```
        i++;
```

```
    } else {
```

```
        linhas[k] = Rlinhas[j];
```

```
        if (tipo == 0) ((int*)coluna)[k] = ((int*)Rcoluna)[j];
```

```
        if (tipo == 1) ((float*)coluna)[k] = ((float*)Rcoluna)[j];
```

```
        if (tipo == 2) ((char**)coluna)[k] = ((char**)Rcoluna)[j];
```

```

        if (tipo == 3) ((long long*)coluna)[k] = ((long long*)Rcoluna)[j];
        j++;
    }
    (*trocas)++; // ? conta a troca
    k++;
}

```

```

while (i < n1) {
    linhas[k] = Llinhas[i];
    if (tipo == 0) ((int*)coluna)[k] = ((int*)Lcoluna)[i];
    if (tipo == 1) ((float*)coluna)[k] = ((float*)Lcoluna)[i];
    if (tipo == 2) ((char**)coluna)[k] = ((char**)Lcoluna)[i];
    if (tipo == 3) ((long long*)coluna)[k] = ((long long*)Lcoluna)[i];
    i++;
    k++;
    (*trocas)++; // ? conta a troca
}

```

```

while (j < n2) {
    linhas[k] = Rlinhas[j];
    if (tipo == 0) ((int*)coluna)[k] = ((int*)Rcoluna)[j];
    if (tipo == 1) ((float*)coluna)[k] = ((float*)Rcoluna)[j];
    if (tipo == 2) ((char**)coluna)[k] = ((char**)Rcoluna)[j];
    if (tipo == 3) ((long long*)coluna)[k] = ((long long*)Rcoluna)[j];
    j++;
    k++;
    (*trocas)++; // ? conta a troca
}

```

```

free(Llinhas);
free(Rlinhas);
free(Lcoluna);
free(Rcoluna);
}

```

```

void mergeSort(char** linhas, void* coluna, int tipo, int esquerda, int direita, int*
trocas) {
    if (esquerda < direita) {
        int meio = esquerda + (direita - esquerda) / 2;
        mergeSort(linhas, coluna, tipo, esquerda, meio, trocas);
        mergeSort(linhas, coluna, tipo, meio + 1, direita, trocas);
        merge(linhas, coluna, tipo, esquerda, meio, direita, trocas);
    }
}

```

```

int MergeSort(char** linhas, void* coluna, int tipo, int n) {
    int trocas = 0;
    mergeSort(linhas, coluna, tipo, 0, n - 1, &trocas);
    return trocas;
}

```

// --- Shell Sort ---

```

int ShellSort(char** linhas, void* coluna, int tipo, int n) {
    int trocas = 0;
    int i, j, gap;

    gap = n / 2;

    while (gap > 0) {
        for (i = gap; i < n; i++) {
            bool trocou = false;

            if (tipo == 0) {
                int temp = ((int*)coluna)[i];
                j = i;
                while (j >= gap && ((int*)coluna)[j - gap] > temp) {

```

```

        ((int*)coluna)[j] = ((int*)coluna)[j - gap];
        trocaLinha(linhas, j, j - gap);
        j -= gap;
        trocas++;
        trocou = true;
    }
    ((int*)coluna)[j] = temp;
}

else if (tipo == 1) {
    float temp = ((float*)coluna)[i];
    j = i;
    while (j >= gap && ((float*)coluna)[j - gap] > temp) {
        ((float*)coluna)[j] = ((float*)coluna)[j - gap];
        trocaLinha(linhas, j, j - gap);
        j -= gap;
        trocas++;
        trocou = true;
    }
    ((float*)coluna)[j] = temp;
}

else if (tipo == 2) {
    char* temp = ((char**)coluna)[i];
    j = i;
    while (j >= gap && strcmp(((char**)coluna)[j - gap], temp) > 0) {
        ((char**)coluna)[j] = ((char**)coluna)[j - gap];
        trocaLinha(linhas, j, j - gap);
        j -= gap;
        trocas++;
        trocou = true;
    }
    ((char**)coluna)[j] = temp;
}

```

```

        else if (tipo == 3) {
            long long temp = ((long long*)coluna)[i];
            j = i;
            while (j >= gap && ((long long*)coluna)[j - gap] > temp) {
                ((long long*)coluna)[j] = ((long long*)coluna)[j - gap];
                trocaLinha(linhas, j, j - gap);
                j -= gap;
                trocas++;
                trocou = true;
            }
            ((long long*)coluna)[j] = temp;
        }
    }
    gap /= 2;
}

return trocas;
}

// --- Heap Sort ---
int heapify(char** linhas, void* coluna, int tipo, int n, int i) {
    int trocas = 0;
    int maior = i;
    int esq = 2 * i + 1;
    int dir = 2 * i + 2;

    if (tipo == 0) {
        if (esq < n && ((int*)coluna)[esq] > ((int*)coluna)[maior]) maior = esq;
        if (dir < n && ((int*)coluna)[dir] > ((int*)coluna)[maior]) maior = dir;
        if (maior != i) {
            int t = ((int*)coluna)[i];

```

```

        ((int*)coluna)[i] = ((int*)coluna)[maior];
        ((int*)coluna)[maior] = t;
        trocaLinha(linhas, i, maior);
        trocas++;
        trocas += heapify(linhas, coluna, tipo, n, maior);
    }
} else if (tipo == 1) {
    if (esq < n && ((float*)coluna)[esq] > ((float*)coluna)[maior]) maior = esq;
    if (dir < n && ((float*)coluna)[dir] > ((float*)coluna)[maior]) maior = dir;
    if (maior != i) {
        float t = ((float*)coluna)[i];
        ((float*)coluna)[i] = ((float*)coluna)[maior];
        ((float*)coluna)[maior] = t;
        trocaLinha(linhas, i, maior);
        trocas++;
        trocas += heapify(linhas, coluna, tipo, n, maior);
    }
} else if (tipo == 2) {
    if (esq < n && strcmp(((char**)coluna)[esq], ((char**)coluna)[maior]) > 0)
maior = esq;
    if (dir < n && strcmp(((char**)coluna)[dir], ((char**)coluna)[maior]) > 0)
maior = dir;
    if (maior != i) {
        char* t = ((char**)coluna)[i];
        ((char**)coluna)[i] = ((char**)coluna)[maior];
        ((char**)coluna)[maior] = t;
        trocaLinha(linhas, i, maior);
        trocas++;
        trocas += heapify(linhas, coluna, tipo, n, maior);
    }
} else if (tipo == 3) {
    if (esq < n && ((long long*)coluna)[esq] > ((long long*)coluna)[maior]) maior
= esq;

```



```

        if (dir < n && ((long long*)coluna)[dir] > ((long long*)coluna)[maior]) maior
= dir;

        if (maior != i) {
            long long t = ((long long*)coluna)[i];
            ((long long*)coluna)[i] = ((long long*)coluna)[maior];
            ((long long*)coluna)[maior] = t;
            trocaLinha(linhas, i, maior);
            trocas++;
            trocas += heapify(linhas, coluna, tipo, n, maior);
        }
    }
    return trocas;
}

```

```

int HeapSort(char** linhas, void* coluna, int tipo, int n) {
    int trocas = 0;
    int i;
    for (i = n / 2 - 1; i >= 0; i--)
        trocas += heapify(linhas, coluna, tipo, n, i);

    for (i = n - 1; i > 0; i--) {
        if (tipo == 0) { int t = ((int*)coluna)[0]; ((int*)coluna)[0] = ((int*)coluna)[i];
        ((int*)coluna)[i] = t; }
        if (tipo == 1) { float t = ((float*)coluna)[0]; ((float*)coluna)[0] =
        ((float*)coluna)[i]; ((float*)coluna)[i] = t; }
        if (tipo == 2) { char* t = ((char**)coluna)[0]; ((char**)coluna)[0] =
        ((char**)coluna)[i]; ((char**)coluna)[i] = t; }
        if (tipo == 3) { long long t = ((long long*)coluna)[0]; ((long long*)coluna)[0]
= ((long long*)coluna)[i]; ((long long*)coluna)[i] = t; }
        trocaLinha(linhas, 0, i);
        trocas++;
        trocas += heapify(linhas, coluna, tipo, i, 0);
    }
}

```

```
    return trocas;
}

int main(void) {
    int opcaoArquivo;
    printf("Escolha o conjunto de dados:\n");
    printf("1 - Dados de 2023\n");
    printf("2 - Dados de 2024\n");
    printf("Opcao: ");
    scanf("%d", &opcaoArquivo);

    const char* arquivo2023 = "focos_br_mg_ref_2023.csv";
    const char* arquivo2024 = "focos_br_mg_ref_2024.csv";

    FILE* arquivo1 = NULL;
    FILE* arquivo2 = NULL;

    switch (opcaoArquivo) {
        case 1:
            arquivo1 = fopen(arquivo2023, "r");
            if (!arquivo1) {
                perror("Erro ao abrir arquivo de 2023");
                return 1;
            }
            break;

        case 2:
            arquivo1 = fopen(arquivo2024, "r");
            if (!arquivo1) {
                perror("Erro ao abrir arquivo de 2024");
                return 1;
            }
            break;
    }
```

```

    default:
        printf("Opcao invalida!\n");
        return 1;
}

```

```

char buffer[65536];
char** linhas = NULL;
int contador = 0;

```

```

while (fgets(buffer, sizeof(buffer), arquivo1)) {
    buffer[strcspn(buffer, "\r\n")] = '\0';
    linhas = (char**)realloc(linhas, (contador + 1) * sizeof(char*));
    linhas[contador] = (char*)malloc(strlen(buffer) + 1);
    strcpy(linhas[contador], buffer);
    contador++;
}
fclose(arquivo1);

```

```

if (arquivo2 != NULL) {
    while (fgets(buffer, sizeof(buffer), arquivo2)) {
        buffer[strcspn(buffer, "\r\n")] = '\0';
        linhas = (char**)realloc(linhas, (contador + 1) * sizeof(char*));
        linhas[contador] = (char*)malloc(strlen(buffer) + 1);
        strcpy(linhas[contador], buffer);
        contador++;
    }
    fclose(arquivo2);
}

```

```

const          char*          nomesColunas[]          =
{"id_bdq","foco_id","lat","lon","data_pas","pais","estado","municipio","bioma"};
int tiposColunas[] = {-1, -1, 1, 1, 3, -1, -1, 2, 2};
int numColunas = 9;

```

```

int escolha;
printf("Escolha o algoritmo de ordenacao:\n");
printf("1 - Selection Sort\n");
printf("2 - Bubble Sort\n");
printf("3 - Insertion Sort\n");
printf("4 - Cocktail Shaker Sort\n");
printf("5 - Quick Sort\n");
printf("6 - Merge Sort\n");
printf("7 - Shell Sort\n");
printf("8 - Heap Sort\n");
printf("Opcao: ");
scanf("%d", &escolha);

char nomeEscolhido[100];
printf("\nColunas ordenaveis: lat, lon, data_pas, municipio, bioma\n");
printf("Digite o nome da coluna: ");
scanf("%99s", nomeEscolhido);

int indiceColuna = -1;
int i;
for (i = 0; i < numColunas; i++) {
    if (strcmp(nomeEscolhido, nomesColunas[i]) == 0) {
        indiceColuna = i;
        break;
    }
}

if (indiceColuna == -1 || tiposColunas[indiceColuna] == -1) {
    printf("Coluna invalida ou nao ordenavel!\n");
    return 1;
}

int tipoColuna = tiposColunas[indiceColuna];

```

```

void* coluna = NULL;
if (tipoColuna == 0) coluna = malloc(contador * sizeof(int));
if (tipoColuna == 1) coluna = malloc(contador * sizeof(float));
if (tipoColuna == 2) coluna = malloc(contador * sizeof(char*));
if (tipoColuna == 3) coluna = malloc(contador * sizeof(long long));

```

```

for (i = 0; i < contador; i++) {
    char* temp = _strdup(linhas[i]);
    char* token = strtok(temp, ",");
    int j;
    for (j = 0; j < indiceColuna; j++)
        token = strtok(NULL, ",");
    if (tipoColuna == 0)
        ((int*)coluna)[i] = atoi(token);
    else if (tipoColuna == 1)
        ((float*)coluna)[i] = atof(token);
    else if (tipoColuna == 2)
        ((char**)coluna)[i] = _strdup(token);
    else if (tipoColuna == 3)
        ((long long*)coluna)[i] = dataParaNumero(token);
    free(temp);
}

```

```
int trocas = 0;
```

```
switch (escolha) {
```

```
    case 1: trocas = SelectionSort(linhas, coluna, tipoColuna, contador);
```

```
break;
```

```
    case 2: trocas = BubbleSort(linhas, coluna, tipoColuna, contador); break;
```

```
    case 3: trocas = InsertionSort(linhas, coluna, tipoColuna, contador); break;
```

```
    case 4: trocas = CocktailShakerSort(linhas, coluna, tipoColuna, contador);
```

```
break;
```

```
    case 5: trocas = QuickSort(linhas, coluna, tipoColuna, contador); break;
```

```
    case 6: trocas = MergeSort(linhas, coluna, tipoColuna, contador); break;
```

```
        case 7: trocas = ShellSort(linhas, coluna, tipoColuna, contador); break;
        case 8: trocas = HeapSort(linhas, coluna, tipoColuna, contador); break;
        default: printf("Opcao invalida!\n"); return 1;
    }

    printf("\n--- Linhas ordenadas ---\n");
    for (i = 0; i < contador; i++) {
        printf("%s\n", linhas[i]);
        free(linhas[i]);
    }

    printf("\nTotal de linhas ordenadas: %d\n", contador);
    printf("Total de trocas realizadas: %d\n", trocas);

    if (tipoColuna == 2) {
        char** arr = (char**)coluna;
        for (i = 0; i < contador; i++)
            free(arr[i]);
    }

    free(coluna);
    free(linhas);

    return 0;
}
```

REFERÊNCIAS

ALMEIDA, M. H. B. de; GOMES, R. C.; ALMEIDA, O. C. P. de; BALLARIN, A. W. Desempenho da técnica Deep Learning na análise e categorização de imagens de defeito de madeira. *Energia na Agricultura*, Botucatu, v. 33, n. 3, p. 284-291, 2018. Disponível em: <http://dx.doi.org/10.17224/EnergAgric.2018v33n3p284-291>. Acesso em: 22 out. 2025.

BAELDUNG. Cocktail Shaker Sort. [S. l.], 2023. Disponível em: <https://www.baeldung.com/cs/cocktail-sort>. Acesso em: 20 out. 2025.

BALIEIRO, Ricardo. Estrutura de Dados. [S. l.], 2023. Disponível em: <https://hood.com.br/new/filegator/repository/Estrutura%20de%20Dados/10%20Livro%20Estrutura%20de%20Dados.pdf>. Acesso em: 19 out. 2025.

FIGUEIRA FILHO, Frederico. QuickSort. Panda IME-USP, [S. l.], 2022. Disponível em: https://panda.ime.usp.br/panda/static/pythonds_pt/05-OrdenacaoBusca/OQuickSort.html. Acesso em: 19 out. 2025.

FORBELLONE, André Luiz Villar; EBERSPÄCHER, Henri Frederico. *Lógica de Programação: A Construção de Algoritmos e Estruturas de Dados*. 3. ed. São Paulo: Pearson Prentice Hall, 2005. Disponível em: <https://books.google.com.br/books>. Acesso em: 18 out. 2025.

GOOGLE. O que é aprendizado não supervisionado? [S. l.], 2024. Disponível em: <https://cloud.google.com/discover/what-is-unsupervised-learning?hl=pt-BR>. Acesso em: 20 out. 2025.

GUARISE, Lucas. Detecção de notícias falsas usando técnicas de deep learning. 2019. 49 f. Monografia (Graduação em Engenharia de Computação) – Instituto de Ciências Matemáticas e de Computação, Universidade de São Paulo, São Carlos, 2019. Acesso em: 23 out. 2025.

HOMEM, William Ludovico. Apostila de Machine Learning. 2020. 46 f. Material didático. Programa de Educação Tutorial - Engenharia Mecânica, Universidade Federal do Espírito Santo, Vitória, 2020. Acesso em: 22 out. 2025.

KUFUNDA. Estruturas de Dados Usando C. [S. l.], 2024. Disponível em: [https://www.kufunda.net/publicdocs/ESTRUTURAS%20DE%20DADOS%20USANDO%20C%20\(AARON%20TENENBAUM,%20YEDIDYAH%20LANGSAM%20etc.\).pdf](https://www.kufunda.net/publicdocs/ESTRUTURAS%20DE%20DADOS%20USANDO%20C%20(AARON%20TENENBAUM,%20YEDIDYAH%20LANGSAM%20etc.).pdf). Acesso em: 20 out. 2025.

LECUN, Yann; BENGIO, Yoshua; HINTON, Geoffrey. Deep learning. Nature, London, v. 521, n. 7553, p. 436-444, maio 2015. DOI: 10.1038/nature14539. Disponível em: <https://www.nature.com/articles/nature14539>. Acesso em: 22 out. 2025.

MENDES, Guilherme. Algoritmos: Merge Sort. Medium, [S. l.], 2021. Disponível em: <https://guilherme-rmendes95.medium.com/algoritmos-merge-sort-ef12dadeba2a>. Acesso em: 19 out. 2025.

PFITZNER, André L.; PINTO, Paulo E. D.; COSTA, Rosa Maria E. M. da. O Algoritmo de Ordenação Smoothsort Explicado. In: ANAIS [...] Instituto de Matemática e Estatística, Universidade do Estado do Rio de Janeiro, Rio de Janeiro, 2009. Acesso em: 21 out. 2025.

PONTI, Moacir A.; COSTA, Gabriel B. Paranhos da. Como funciona o Deep Learning. In: Tópicos em Gerenciamento de Dados e Informações. 1. ed. Porto Alegre: Sociedade Brasileira de Computação, 2017. p. 65-93. ISBN 978-85-7669-400-7. Acesso em: 22 out. 2025.

PONTES, Britto; COSTA, Anna. Como Funciona o Deep Learning. 2017. Disponível em: https://sites.icmc.usp.br/moacir/papers/Ponti_Costa_Como-funciona-o-Deep-Learning_2017.pdf. Acesso em: 20 out. 2025.

PREATTI, Clayton L. et al. A Importância do Ensino de Estruturas de Dados na Formação em Computação. Revista Edufatec, Franca, v. 5, n. 2, p. 45-58, 2023.

Disponível em: <https://revistaedufatec.fatecfranca.edu.br/wp-content/uploads/2023/04/edufatec-n05v2a05.pdf>. Acesso em: 21 out. 2025.

Quick Sort. [S. l.], 2022. Disponível em: <https://joaoarthurbm.github.io/eda/posts/quick-sort>. Acesso em: 20 out. 2025.

PROGRAMIZ. Shell Sort. [S. l.], 2024. Disponível em: https://www-programiz-com.translate.goog/dsa/shell-sort?_x_tr_sl=en&_x_tr_tl=pt&_x_tr_hl=pt&_x_tr_pto=tc. Acesso em: 19 out. 2025.

SANTOS, Antônio. Estrutura de Dados em Linguagem C. [S. l.], 2024. Disponível em: <https://www.dio.me/articles/estrutura-de-dados-em-linguagem-c>. Acesso em: 18 out. 2025.

Shell Sort. Panda IME-USP, [S. l.], 2022. Disponível em: https://panda.ime.usp.br/panda/static/pythonds_pt/05-OrdenacaoBusca/OShellSort.html. Acesso em: 20 out. 2025.

SHARMA, Nisha. What is Machine Learning? Berkeley School of Information, [S. l.], 2021. Disponível em: <https://ischoolonline.berkeley.edu/blog/what-is-machine-learning/>. Acesso em: 20 out. 2025.

SILVA, João Arthur. Merge Sort. [S. l.], 2022. Disponível em: <https://joaoarthurbm.github.io/eda/posts/merge-sort>. Acesso em: 20 out. 2025.

SILVA, João Arthur Brunet Monteiro. Selection Sort. [S. l.], 2022. Disponível em: <https://joaoarthurbm.github.io/eda/posts/selection-sort/>. Acesso em: 21 out. 2025.

SOUSA, Jackson É. G.; RICARTE, João V. G.; LIMA, Náthalee C. A. Algoritmos de ordenação: um estudo comparativo. In: ENCONTRO DE COMPUTAÇÃO DO OESTE POTIGUAR – ECOP/UFERSA, 2017, Pau dos Ferros. Anais [...]. Pau dos Ferros: UFERSA, 2017. v. 1, p. 166-173. Disponível em: <https://periodicos.ufersa.edu.br/index.php/ecop/article/view/>. Acesso em: 01 nov. 2025.

TENENBAUM, Aaron M.; LANGSAM, Yedidiah; AUGENSTEIN, Moshe J. Estruturas de Dados Usando C. São Paulo: Pearson Makron Books, 1995. Disponível em: https://d1wqtxts1xzle7.cloudfront.net/30666289/Apostila_-_Estrutura_de_dados-libre.pdf?1391811480=&response-content-disposition=inline%3B+filename%3DEstruturas_de_Dados.pdf. Acesso em: 21 out. 2025.

W3SCHOOLS. Data Structures and Algorithms Selection Sort. [S. l.], 2024. Disponível em: https://www.w3schools.com/dsa/dsa_algo_selectionsort.php. Acesso em: 1 nov. 2025.

ZHAO, Sherry. Supervised vs. Unsupervised Learning: What's the Difference? NVIDIA Blog, [S. l.], 2021. Disponível em: <https://blogs.nvidia.com/blog/supervised-unsupervised-learning/>. Acesso em: 20 out. 2025.

8 FICHAS DE ATIVIDADES PRÁTICAS SUPERVISIONADAS

UNIP UNIVERSIDADE PAULISTA		FICHA DAS ATIVIDADES PRÁTICAS SUPERVISIONADAS - APS			
NOME: Amanda Justino Oliveira		TURMA: CC4A68		RA: R093689	
CURSO: Ciências da Computação		CAMPUS: Paraíso		SEMESTRE: 4º	
CÓDIGO DA ATIVIDADE: 77B1		SEMESTRE: 4º		ANO GRADE: 2025	
TURN O: Matutino					
DATA DA ATIVIDADE	DESCRIÇÃO DA ATIVIDADE	TOTAL DE HORAS	ASSINATURA DO ALUNO	HORAS ATRIBUÍDAS (1)	ASSINATURA DO PROFESSOR
11/09	Plano de desenvolvimento do projeto	3hrs	Amanda Justino Oliveira		
13/09	Pesquisa sobre conceitos de estruturas de dados	5hrs	Amanda Justino Oliveira		
18/09	Pesquisa sobre bubble sort e insertion sort	4hrs	Amanda Justino Oliveira		
19/09	Pesquisa sobre selection sort, quick sort, merge sort	4hrs	Amanda Justino Oliveira		
26/09	Pesquisa sobre shell sort, cocktail Shaker sort, heap sort	4hrs	Amanda Justino Oliveira		
27/09	Pesquisa sobre machine learning, deep learning	4hrs	Amanda Justino Oliveira		
09/10	Pesquisa sobre técnicas de machine learning e deep learning	5hrs	Amanda Justino Oliveira		
12/10	Síntese das Pesquisas para elaboração do documento ABNT	8hrs	Amanda Justino Oliveira		
16/10	Desenvolvimento dos metodos de ordenação	8hrs	Amanda Justino Oliveira		
18/10	Aprimoramento metodos de ordenação	7hrs	Amanda Justino Oliveira		
25/10	Testes dos metodos de ordenação	3hrs	Amanda Justino Oliveira		
26/10	Tratamento de erros	4hrs	Amanda Justino Oliveira		
30/10	Escrita do Documento ABNT (Introdução, Referencial Teórico)	8hrs	Amanda Justino Oliveira		
02/11	Escrita final do Documento ABNT (Desenvolvimento, Conclusão)	8hrs	Amanda Justino Oliveira		
08/11	Revisão do documento ABNT	3hrs	Amanda Justino Oliveira		
11/11	Revisão Final	2hrs	Amanda Justino Oliveira		
(1) Horas atribuídas de acordo com o regulamento das Atividades Práticas Supervisionadas do curso.			TOTAL DE HORAS ATRIBUÍDAS: 80Hrs		
			AVALIAÇÃO: _____ Aprovado ou Reprovado		
			NOTA: _____		
			DATA: ____/____/____		
CARIMBO E ASSINATURA DO COORDENADOR DO CURSO					

UNIP UNIVERSIDADE PAULISTA		FICHA DAS ATIVIDADES PRÁTICAS SUPERVISIONADAS - APS			
NOME: Bruno Alves de Sousa		TURMA: CC4A68		RA: G975BG3	
CURSO: Ciências da Computação		CAMPUS: Paraíso		SEMESTRE: 4º	
CÓDIGO DA ATIVIDADE: 77B1		SEMESTRE: 4º		ANO GRADE: 2025	
TURN O: Matutino					
DATA DA ATIVIDADE	DESCRIÇÃO DA ATIVIDADE	TOTAL DE HORAS	ASSINATURA DO ALUNO	HORAS ATRIBUÍDAS (1)	ASSINATURA DO PROFESSOR
11/09	Plano de desenvolvimento do projeto	3hrs	Bruno A. Saldou		
13/09	Pesquisa sobre conceitos de estruturas de dados	5hrs	Bruno A. Saldou		
18/09	Pesquisa sobre bubble sort e insertion sort	4hrs	Bruno A. Saldou		
19/09	Pesquisa sobre selection sort, quick sort, merge sort	4hrs	Bruno A. Saldou		
26/09	Pesquisa sobre shell sort, cocktail Shaker sort, heap sort	4hrs	Bruno A. Saldou		
27/09	Pesquisa sobre machine learning, deep learning	4hrs	Bruno A. Saldou		
09/10	Pesquisa sobre técnicas de machine learning e deep learning	5hrs	Bruno A. Saldou		
12/10	Síntese das Pesquisas para elaboração do documento ABNT	8hrs	Bruno A. Saldou		
16/10	Desenvolvimento dos metodos de ordenação	8hrs	Bruno A. Saldou		
18/10	Aprimoramento metodos de ordenação	7hrs	Bruno A. Saldou		
25/10	Testes dos metodos de ordenação	3hrs	Bruno A. Saldou		
26/10	Tratamento de erros	4hrs	Bruno A. Saldou		
30/10	Escrita do Documento ABNT (Introdução, Referencial Teórico)	8hrs	Bruno A. Saldou		
02/11	Escrita final do Documento ABNT (Desenvolvimento, Conclusão)	8hrs	Bruno A. Saldou		
08/11	Revisão do documento ABNT	3hrs	Bruno A. Saldou		
11/11	Revisão Final	2hrs	Bruno A. Saldou		
(1) Horas atribuídas de acordo com o regulamento das Atividades Práticas Supervisionadas do curso.			TOTAL DE HORAS ATRIBUÍDAS: 80Hrs		
			AVALIAÇÃO: _____ Aprovado ou Reprovado		
			NOTA: _____		
			DATA: ____/____/____		
CARIMBO E ASSINATURA DO COORDENADOR DO CURSO					

UNIP		FICHA DAS ATIVIDADES PRÁTICAS SUPERVISIONADAS - APS			
UNIVERSIDADE PAULISTA					
NOME: Breno Faustino Almeida		TURMA: CC4A68		RA: G972732	
CURSO: Ciências da Computação		CAMPUS: Paraíso		SEMESTRE: 4º	
CÓDIGO DA ATIVIDADE: 77B1		SEMESTRE: 4º		ANO GRADE: 2025	
TURN: Matutino					
DATA DA ATIVIDADE	DESCRIÇÃO DA ATIVIDADE	TOTAL DE HORAS	ASSINATURA DO ALUNO	HORAS ATRIBUÍDAS (1)	ASSINATURA DO PROFESSOR
11/09	Plano de desenvolvimento do projeto	3hrs	Breno Faustino Almeida		
13/09	Pesquisa sobre conceitos de estruturas de dados	5hrs	Breno Faustino Almeida		
18/09	Pesquisa sobre bubble sort e insertion sort	4hrs	Breno Faustino Almeida		
19/09	Pesquisa sobre selection sort, quick sort, merge sort	4hrs	Breno Faustino Almeida		
26/09	Pesquisa sobre shell sort, cocktail Shaker sort, heap sort	4hrs	Breno Faustino Almeida		
27/09	Pesquisa sobre machine learning, deep learning	4hrs	Breno Faustino Almeida		
09/10	Pesquisa sobre técnicas de machine learning e deep learning	5hrs	Breno Faustino Almeida		
12/10	Síntese das Pesquisas para elaboração do documento ABNT	8hrs	Breno Faustino Almeida		
16/10	Desenvolvimento dos metodos de ordenação	8hrs	Breno Faustino Almeida		
18/10	Aprimoramento metodos de ordenação	7hrs	Breno Faustino Almeida		
25/10	Testes dos metodos de ordenação	3hrs	Breno Faustino Almeida		
26/10	Tratamento de erros	4hrs	Breno Faustino Almeida		
30/10	Escrita do Documento ABNT (Introdução, Referencial Teórico)	8hrs	Breno Faustino Almeida		
02/11	Escrita final do Documento ABNT (Desenvolvimento, Conclusão)	8hrs	Breno Faustino Almeida		
08/11	Revisão do documento ABNT	3hrs	Breno Faustino Almeida		
11/11	Revisão Final	2hrs	Breno Faustino Almeida		
(1) Horas atribuídas de acordo com o regulamento das Atividades Práticas Supervisionadas do curso.			TOTAL DE HORAS ATRIBUÍDAS: 80Hrs		
			AVALIAÇÃO: _____		
			Aprovado ou Reprovado		
			NOTA: _____		
			DATA: ____/____/____		
			CARIMBO E ASSINATURA DO COORDENADOR DO CURSO		

UNIP		FICHA DAS ATIVIDADES PRÁTICAS SUPERVISIONADAS - APS			
UNIVERSIDADE PAULISTA					
NOME: Gabriel Santana Cordeiro		TURMA: CC4A68		RA: G972716	
CURSO: Ciências da Computação		CAMPUS: Paraíso		SEMESTRE: 4º	
CÓDIGO DA ATIVIDADE: 77B1		SEMESTRE: 4º		ANO GRADE: 2025	
TURN: Matutino					
DATA DA ATIVIDADE	DESCRIÇÃO DA ATIVIDADE	TOTAL DE HORAS	ASSINATURA DO ALUNO	HORAS ATRIBUÍDAS (1)	ASSINATURA DO PROFESSOR
11/09	Plano de desenvolvimento do projeto	3hrs	Gabriel Santana Cordeiro		
13/09	Pesquisa sobre conceitos de estruturas de dados	5hrs	Gabriel Santana Cordeiro		
18/09	Pesquisa sobre bubble sort e insertion sort	4hrs	Gabriel Santana Cordeiro		
19/09	Pesquisa sobre selection sort, quick sort, merge sort	4hrs	Gabriel Santana Cordeiro		
26/09	Pesquisa sobre shell sort, cocktail Shaker sort, heap sort	4hrs	Gabriel Santana Cordeiro		
27/09	Pesquisa sobre machine learning, deep learning	4hrs	Gabriel Santana Cordeiro		
09/10	Pesquisa sobre técnicas de machine learning e deep learning	5hrs	Gabriel Santana Cordeiro		
12/10	Síntese das Pesquisas para elaboração do documento ABNT	8hrs	Gabriel Santana Cordeiro		
16/10	Desenvolvimento dos metodos de ordenação	8hrs	Gabriel Santana Cordeiro		
18/10	Aprimoramento metodos de ordenação	7hrs	Gabriel Santana Cordeiro		
25/10	Testes dos metodos de ordenação	3hrs	Gabriel Santana Cordeiro		
26/10	Tratamento de erros	4hrs	Gabriel Santana Cordeiro		
30/10	Escrita do Documento ABNT (Introdução, Referencial Teórico)	8hrs	Gabriel Santana Cordeiro		
02/11	Escrita final do Documento ABNT (Desenvolvimento, Conclusão)	8hrs	Gabriel Santana Cordeiro		
08/11	Revisão do documento ABNT	3hrs	Gabriel Santana Cordeiro		
11/11	Revisão Final	2hrs	Gabriel Santana Cordeiro		
(1) Horas atribuídas de acordo com o regulamento das Atividades Práticas Supervisionadas do curso.			TOTAL DE HORAS ATRIBUÍDAS: 80Hrs		
			AVALIAÇÃO: _____		
			Aprovado ou Reprovado		
			NOTA: _____		
			DATA: ____/____/____		
			CARIMBO E ASSINATURA DO COORDENADOR DO CURSO		

UNIP		FICHA DAS ATIVIDADES PRÁTICAS SUPERVISIONADAS - APS			
UNIVERSIDADE PAULISTA					
NOME: Maria Eduarda Moreira		TURMA: CC4A68		RA: R0726G9	
CURSO: Ciências da Computação		CAMPUS: Paraíso		SEMESTRE: 4º	
CÓDIGO DA ATIVIDADE: 77B1		SEMESTRE: 4º		ANO GRADE: 2025	
TURNQ: Matutino					
DATA DA ATIVIDADE	DESCRIÇÃO DA ATIVIDADE	TOTAL DE HORAS	ASSINATURA DO ALUNO	HORAS ATRIBUÍDAS (1)	ASSINATURA DO PROFESSOR
11/09	Plano de desenvolvimento do projeto	3hrs	<i>Maria Eduarda</i>		
13/09	Pesquisa sobre conceitos de estruturas de dados	5hrs	<i>Maria Eduarda</i>		
18/09	Pesquisa sobre bubble sort e insertion sort	4hrs	<i>Maria Eduarda</i>		
19/09	Pesquisa sobre selection sort, quick sort, merge sort	4hrs	<i>Maria Eduarda</i>		
26/09	Pesquisa sobre shell sort, cocktail Shaker sort, heap sort	4hrs	<i>Maria Eduarda</i>		
27/09	Pesquisa sobre machine learning, deep learning	4hrs	<i>Maria Eduarda</i>		
09/10	Pesquisa sobre técnicas de machine learning e deep learning	5hrs	<i>Maria Eduarda</i>		
12/10	Síntese das Pesquisas para elaboração do documento ABNT	8hrs	<i>Maria Eduarda</i>		
16/10	Desenvolvimento dos metodos de ordenação	8hrs	<i>Maria Eduarda</i>		
18/10	Aprimoramento metodos de ordenação	7hrs	<i>Maria Eduarda</i>		
25/10	Testes dos metodos de ordenação	3hrs	<i>Maria Eduarda</i>		
26/10	Tratamento de erros	4hrs	<i>Maria Eduarda</i>		
30/10	Escrita do Documento ABNT (Introdução, Referencial Teórico)	8hrs	<i>Maria Eduarda</i>		
02/11	Escrita final do Documento ABNT (Desenvolvimento, Conclusão)	8hrs	<i>Maria Eduarda</i>		
08/11	Revisão do documento ABNT	3hrs	<i>Maria Eduarda</i>		
11/11	Revisão Final	2hrs	<i>Maria Eduarda</i>		
(1) Horas atribuídas de acordo com o regulamento das Atividades Práticas Supervisionadas do curso.			TOTAL DE HORAS ATRIBUÍDAS: 80Hrs		
			AVALIAÇÃO: _____		
			Aprovado ou Reprovado		
			NOTA: _____		
			DATA: ____/____/____		
			CARIMBO E ASSINATURA DO COORDENADOR DO CURSO		

UNIP		FICHA DAS ATIVIDADES PRÁTICAS SUPERVISIONADAS - APS			
UNIVERSIDADE PAULISTA					
NOME: Pedro Henrique Gomes dos Santos		TURMA: CC4A68		RA: R085287	
CURSO: Ciências da Computação		CAMPUS: Paraíso		SEMESTRE: 4º	
CÓDIGO DA ATIVIDADE: 77B1		SEMESTRE: 4º		ANO GRADE: 2025	
TURNQ: Matutino					
DATA DA ATIVIDADE	DESCRIÇÃO DA ATIVIDADE	TOTAL DE HORAS	ASSINATURA DO ALUNO	HORAS ATRIBUÍDAS (1)	ASSINATURA DO PROFESSOR
11/09	Plano de desenvolvimento do projeto	3hrs	<i>Pedro Henrique</i>		
13/09	Pesquisa sobre conceitos de estruturas de dados	5hrs	<i>Pedro Henrique</i>		
18/09	Pesquisa sobre bubble sort e insertion sort	4hrs	<i>Pedro Henrique</i>		
19/09	Pesquisa sobre selection sort, quick sort, merge sort	4hrs	<i>Pedro Henrique</i>		
26/09	Pesquisa sobre shell sort, cocktail Shaker sort, heap sort	4hrs	<i>Pedro Henrique</i>		
27/09	Pesquisa sobre machine learning, deep learning	4hrs	<i>Pedro Henrique</i>		
09/10	Pesquisa sobre técnicas de machine learning e deep learning	5hrs	<i>Pedro Henrique</i>		
12/10	Síntese das Pesquisas para elaboração do documento ABNT	8hrs	<i>Pedro Henrique</i>		
16/10	Desenvolvimento dos metodos de ordenação	8hrs	<i>Pedro Henrique</i>		
18/10	Aprimoramento metodos de ordenação	7hrs	<i>Pedro Henrique</i>		
25/10	Testes dos metodos de ordenação	3hrs	<i>Pedro Henrique</i>		
26/10	Tratamento de erros	4hrs	<i>Pedro Henrique</i>		
30/10	Escrita do Documento ABNT (Introdução, Referencial Teórico)	8hrs	<i>Pedro Henrique</i>		
02/11	Escrita final do Documento ABNT (Desenvolvimento, Conclusão)	8hrs	<i>Pedro Henrique</i>		
08/11	Revisão do documento ABNT	3hrs	<i>Pedro Henrique</i>		
11/11	Revisão Final	2hrs	<i>Pedro Henrique</i>		
(1) Horas atribuídas de acordo com o regulamento das Atividades Práticas Supervisionadas do curso.			TOTAL DE HORAS ATRIBUÍDAS: 80Hrs		
			AVALIAÇÃO: _____		
			Aprovado ou Reprovado		
			NOTA: _____		
			DATA: ____/____/____		
			CARIMBO E ASSINATURA DO COORDENADOR DO CURSO		